

★ QUICK 2 KNOWLEDGE ★

Trang: 1

Ngày: __/__/__

CẨM NANG TẤT CẢ CÁC PATTERN DSA

Những Điều Bạn Cần Biết Trước Khi Phỏng Vấn



PHƯƠNG PHÁP QUAN TRỌNG

Kiên Trì
Bền Bì



★ QUICK 2 KNOWLEDGE ★

CẨM NANG TẤT CẢ CÁC PATTERN DSA

Những Điều Bạn Cần Biết Trước Khi Phỏng Vấn

Trang: 2

Ngày: __/__/__



★ 1. Arrays (Pattern Quan Trọng Nhất) ✓ Hãy nắm vững những bài này!

Pattern / Problem	Example	Code (Java)	Time & Space
1 Two Sum Cho mảng nums và target , trả về chỉ số của hai số sao cho tổng của chúng bằng target . Đầu vào: [2,1,1,15], target =9 Đầu ra: [0,1]	☒ Dùng HashMap ☒ Lưu phần tử và chỉ số của nó ☒ Với mỗi phần tử, kiểm tra (target - current) có tồn tại trong map không	<pre>public int[] twoSum(int[] nums, int target) { Map<Integer, Integer> map = new HashMap<>(); for (int i = 0; i < nums.length; i++) { int rem = target - nums[i]; if (map.containsKey(rem)) { return new int[]{map.get(rem), i}; } map.put(nums[i], i); } return new int[0]; }</pre>	Time: O(n) Space: O(n) Vì sao? Ta chỉ duyệt mảng một lần và dùng HashMap để tra cứu.
2 Best Time to Buy and Sell Stock Tìm lợi nhuận tối đa khi mua và bán một lần. Đầu vào: [7,1,5,3,6,4] Đầu ra: 5	☒ Theo dõi giá nhỏ nhất tính đến hiện tại ☒ Tính lợi nhuận lớn nhất trong khi duyệt	<pre>public int maxProfit(int[] prices) { int min = Integer.MAX_VALUE, max = 0; for (int price : prices) { min = Math.min(min, price); max = Math.max(max, price - min); } return max; }</pre>	Time: O(n) Space: O(1) Vì sao? Duyệt một lần với các biến hằng số.
3 Maximum Subarray (Kadane's Algorithm) Tìm subarray liên tiếp có tổng lớn nhất. Đầu vào: [-2,1,-3,4,-1,2,1,-5,4] Đầu ra: 6 (4,-1,2,1)	☒ Dùng Kadane's Algo ☒ Lưu tổng hiện tại và tổng lớn nhất ☒ Nếu tổng hiện tại < 0, reset về 0	<pre>public int maxSubArray(int[] nums) { int maxSoFar = nums[0], curr = 0; for (int num : nums) { curr = Math.max(num, curr + num); maxSoFar = Math.max(maxSoFar, curr); } return maxSoFar; }</pre>	Time: O(n) Space: O(1) Vì sao? Duyệt một lần, không gian hằng số.
4 Rotate Array by K Xoay mảng sang phải k bước. Đầu vào: [1,2,3,4,5,6,7], k=3 Đầu ra: [5,6,7,1,2,3,4]	1. Đảo ngược toàn bộ mảng 2. Đảo ngược k phần tử đầu 3. Đảo ngược phần còn lại	<pre>public void rotate(int[] nums, int k) { k = k % nums.length; reverse(nums, 0, nums.length - 1); reverse(nums, 0, k - 1); reverse(nums, k, nums.length - 1); } private void reverse(int[] nums, int l, int r) { while (l < r) { int t = nums[l]; nums[l] = nums[r]; nums[r] = t; l++; r--; } }</pre>	Time: O(n) Space: O(1)
5 Remove Duplicates from Sorted Array Xóa phần tử trùng tại chỗ và trả về độ dài mới. Đầu vào: [1,1,2,2,3,3,4] Đầu ra: 4, Array= [1,2,3,4]	☒ Dùng Two Pointers ☒ Con trỏ chậm → phần tử duy nhất ☒ Con trỏ nhanh → duyệt mảng	<pre>public int removeDuplicates(int[] nums) { if (nums.length == 0) return 0; int i = 0; for (int j = 1; j < nums.length; j++) { if (nums[j] != nums[i]) nums[++i] = nums[j]; } return i + 1; }</pre>	Time: O(n) Space: O(1)
6 Merge Intervals Gộp tất cả các interval chồng lẫn nhau. Đầu vào: [[1,3],[2,6],[8,10],[15,18]] Đầu ra: [[1,6],[8,10],[15,18]]	1. Sắp xếp interval theo thời điểm bắt đầu 2. So sánh & gộp những cái chồng lẫn	<pre>Arrays.sort(intervals, (a,b) -> a[0] - b[0]); List<int[]> res = new ArrayList<>(); res.add(intervals[0]); for (int[] in : intervals) { int[] last = res.get(res.size()-1); if (in[0] <= last[1]) last[1] = Math.max(last[1], in[1]); else res.add(in); } return res.toArray(new int[res.size()][2]);</pre>	Time: O(n log n) Space: O(n) Vì sao? Sắp xếp mất n log n.



★ QUICK 2 KNOWLEDGE ★

CẨM NANG TẤT CẢ CÁC PATTERN DSA

Những Điều Bạn Cần Biết Trước Khi Phỏng Vấn

Trang: 3

Ngày: __/__/__



Pattern / Problem	Example	Code (Java)	Time & Space
7 Sliding Window (Fixed Size) Tìm tổng lớn nhất của subarray có kích thước k. Đầu vào: [2,1,5,1,3,2], k=3 Đầu ra: 9 (5,1,3)	☑ Tổng lớn nhất của subarray kích thước k ☑ Số âm đầu tiên trong window kích thước k ☑ Đếm anagram trong chuỗi	<pre>public int maxSum(int[] nums, int k) { int windowSum = 0, maxSum = Integer.MIN_VALUE; for (int i = 0; i < k; i++) windowSum += nums[i]; maxSum = windowSum; for (int i = k; i < nums.length; i++) { windowSum += nums[i] - nums[i - k]; maxSum = Math.max(maxSum, windowSum); } return maxSum; }</pre>	Time: O(n) Space: O(1) Vì sao? Mỗi phần tử được xét tối đa hai lần.
8 Kadane's Algorithm (Max Subarray Sum) Tìm subarray liên tiếp có tổng lớn nhất. Đầu vào: [-2,1,-3,4,-1,2,1,-5,4] Đầu ra: 6 (4,-1,2,1)	☑ Tổng subarray lớn nhất ☑ Tổng subarray vòng lớn nhất ☑ Tổng lớn nhất với độ dài tối thiểu K	<pre>public int maxSubArray(int[] nums) { int maxSoFar = nums[0], curr = nums[0]; for (int i = 1; i < nums.length; i++) { curr = Math.max(nums[i], curr + nums[i]); maxSoFar = Math.max(maxSoFar, curr); } return maxSoFar; }</pre>	Time: O(n) Space: O(1) Vì sao? Duyệt một lần, không gian hằng số.
9 Two Pointers Cho mảng đã sắp xếp và target, tìm xem có cặp nào có tổng bằng target không. Đầu vào: [1,2,3,4,6], target=6 Đầu ra: (1,5)	☑ Tổng cặp trong mảng đã sắp xếp ☑ Xóa phần tử trùng trong mảng đã sắp xếp ☑ Bài toán 3-sum	<pre>public boolean pairSum(int[] arr, int target) { int left = 0, right = arr.length - 1; while (left < right) { int sum = arr[left] + arr[right]; if (sum == target) return true; else if (sum < target) left++; else right--; } return false; }</pre>	Time: O(n) Space: O(1) Vì sao? Mỗi phần tử được xét tối đa một lần.
10 Binary Search Tìm một phần tử trong mảng đã sắp xếp. Đầu vào: [1,3,5,7,9,11], target=7 Đầu ra: 3 (chỉ số bắt đầu từ 0)	☑ Binary search ☑ Vị trí xuất hiện đầu/cuối ☑ Tìm kiếm trong mảng đã xoay	<pre>public int binarySearch(int[] arr, int target) { int low = 0, high = arr.length - 1; while (low <= high) { int mid = low + (high - low) / 2; if (arr[mid] == target) return mid; else if (arr[mid] < target) low = mid + 1; else high = mid - 1; } return -1; }</pre>	Time: O(log n) Space: O(1) Vì sao? Không gian tìm kiếm giảm một nửa sau mỗi bước.
11 Merge Two Sorted Arrays Gộp hai mảng đã sắp xếp thành một mảng đã sắp xếp. Đầu vào: [1,3,5], [2,4,6] Đầu ra: [1,2,3,4,5,6]	☑ Gộp hai mảng đã sắp xếp ☑ Gộp k danh sách đã sắp xếp ☑ Gộp các interval	<pre>public int[] merge(int[] a, int[] b) { int n = a.length, m = b.length; int[] res = new int[n + m]; int i = 0, j = 0, k = 0; while (i < n && j < m) { res[k++] = (a[i] <= b[j]) ? a[i++] : b[j++]; } while (i < n) res[k++] = a[i++]; while (j < m) res[k++] = b[j++]; return res; }</pre>	Time: O(n+m) Space: O(n+m) Vì sao? Mỗi phần tử chỉ được copy một lần.
12 Fast & Slow Pointer (Tortoise & Hare) Phát hiện chu trình trong linked list. Đầu vào: 1 → 2 → 3 → 4 → 2 ... Đầu ra: Cycle exists	☑ Phát hiện chu trình trong LL ☑ Tìm điểm giữa của LL ☑ Bài toán Happy number	<pre>public boolean hasCycle(ListNode head) { if (head == null head.next == null) return false; ListNode slow = head, fast = head.next; while (fast != null && fast.next != null) { if (slow == fast) return true; slow = slow.next; fast = fast.next.next; } return false; }</pre>	Time: O(n) Space: O(1) Vì sao? Con trỏ nhanh di chuyển 2x, chậm di chuyển 1x.



★ QUICK 2 KNOWLEDGE ★

CẨM NANG TẤT CẢ CÁC PATTERN DSA

Những Điều Bạn Cần Biết Trước Khi Phỏng Vấn

Trang: 4

Ngày: __/__/__



Pattern / Problem	Example	Code (Java)	Time & Space
13 Longest Substring Without Repeating Characters Tìm độ dài substring dài nhất không có ký tự lặp lại. Đầu vào: s = "abcacbb" Đầu ra: 3 ("abc")	- Sliding window - Dùng HashMap / HashSet - Mở rộng window & thu hẹp khi gặp trùng	<pre>public int lengthOfLongestSubstring(String s) { Map<Character, Integer> map = new HashMap<>(); int left = 0, max = 0; for (int right = 0; right < s.length(); right++) { char c = s.charAt(right); if (map.containsKey(c) && map.get(c) >= left) left = map.get(c) + 1; map.put(c, right); max = Math.max(max, right - left + 1); } return max; }</pre>	Time: O(n) Space: O(k) (k = số ký tự khác nhau) Vi sao? Mỗi ký tự được xét tối đa hai lần.
14 Longest Palindromic Substring Tìm substring dài nhất là palindrome. Đầu vào: s = "babad" Đầu ra: "bab" hoặc "aba"	- Mở rộng từ trung tâm (Expand Around Center) - Palindrome độ dài chẵn & lẻ - Lưu độ dài lớn nhất	<pre>public String longestPalindrome(String s) { if (s.isEmpty()) return ""; int start = 0, maxLen = 1; for (int i = 0; i < s.length(); i++) { int len1 = expand(s, i, i); int len2 = expand(s, i, i + 1); int len = Math.max(len1, len2); if (len > maxLen) { maxLen = len; start = i - (len - 1) / 2; } } return s.substring(start, start + maxLen); } private int expand(String s, int l, int r) { while (l >= 0 && r < s.length() && s.charAt(l) == s.charAt(r)) { l--; r++; } return r - l - 1; }</pre>	Time: O(n²) Space: O(1) Vi sao? Với mỗi trung tâm, ta mở rộng ra hai bên.
15 Group Anagrams Nhóm các từ là anagram của nhau. Đầu vào: ["eat", "tea", "tan", "ate", "nat", "bat"] Đầu ra: [["eat", "tea", "ate"], ["tan", "nat", "bat"]]	- Sắp xếp ký tự làm key - Dùng HashMap<String, List<String>> - Nhóm theo key	<pre>public List<List<String>> groupAnagrams(String[] strs) { Map<String, List<String>> map = new HashMap<>(); for (String s : strs) { char[] arr = s.toCharArray(); Arrays.sort(arr); String key = new String(arr); map.computeIfAbsent(key, k -> new ArrayList<>()).add(s); } return new ArrayList<>(map.values()); }</pre>	Time: O(n*k log k) (k = độ dài chuỗi lớn nhất) Space: O(n*k) Vi sao? Sắp xếp mỗi chuỗi tốn k log k.
16 Top K Frequent Elements Tìm k phần tử xuất hiện nhiều nhất. Đầu vào: nums=[1,1,1,2,3], k=2 Đầu ra: [1,2]	- Đếm tần suất bằng HashMap - Dùng Min Heap kích thước k - Giữ lại k phần tử lớn nhất	<pre>public int[] topKFrequent(int[] nums, int k) { Map<Integer, Integer> map = new HashMap<>(); for (int n : nums) map.put(n, map.getOrDefault(n, 0) + 1); PriorityQueue<int> pq = new PriorityQueue<>((a, b) -> a[1] - b[1]); for (int key : map.keySet()) { pq.offer(new int[]{key, map.get(key)}); if (pq.size() > k) pq.poll(); } int[] res = new int[k], i = k - 1; while (!pq.isEmpty()) res[i--] = pq.poll()[0]; return res; }</pre>	Time: O(n log k) Space: O(n+k) Vi sao? Ta giữ kích thước heap tối đa là k.
17 Kth Largest Element in Array Tìm phần tử lớn thứ k. Đầu vào: nums=[3,2,1,5,6,4], k=2 Đầu ra: 5	- Dùng Min Heap kích thước k - Hoặc QuickSelect (Partition) - Hiệu quả cho mảng lớn	<pre>public int findKthLargest(int[] nums, int k) { PriorityQueue<Integer> pq = new PriorityQueue<>(); for (int n : nums) { pq.offer(n); if (pq.size() > k) pq.poll(); } return pq.peek(); }</pre>	Time: O(n log k) Space: O(k) Vi sao? Kích thước heap giới hạn ở mức k.
18 Product of Array Except Self Trả về mảng mà ans[i] là tích của tất cả phần tử trừ nums[i]. Đầu vào: [1,2,3,4] Đầu ra: [24,12,8,6]	- Dùng tích prefix & suffix - Không dùng phép chia - Không gian phụ O(1)	<pre>public int[] productExceptSelf(int[] nums) { int n = nums.length; int[] res = new int[n]; res[0] = 1; for (int i = 1; i < n; i++) res[i] = res[i-1] * nums[i-1]; int right = 1; for (int i = n - 1; i >= 0; i--) { res[i] *= right; right *= nums[i]; } return res; }</pre>	Time: O(n) Space: O(1) (ngại trừ mảng output) Vi sao? Ta dùng tích prefix và suffix để tránh phép chia.

★ QUICK 2 KNOWLEDGE ★

CẨM NANG TẤT CẢ CÁC PATTERN DSA

Những Điều Bạn Cần Biết Trước Khi Phỏng Vấn

Trang: 5

Ngày: _/_/__



Pattern / Problem	Example	Code (Java)	Time & Space
19 Subarray with Given Sum (K) Tìm một subarray có tổng bằng k. Đầu vào: nums=[1,2,3,7,5], k=12 Đầu ra: (2,4) → [3,7,5]	☒ Prefix Sum + HashMap ☒ Lưu chỉ số của tổng prefix ☒ Kiểm tra (sum - k) có tồn tại không	<pre>public int[] subarraySum(int[] nums, int k) { Map<Integer, Integer> map = new HashMap<>(); int sum = 0; map.put(0, -1); // sum -> index for (int i = 0; i < nums.length; i++) { sum += nums[i]; if (map.containsKey(sum - k)) { return new int[]{map.get(sum - k) + 1, i}; } map.put(sum, i); } return new int[]{-1, -1}; }</pre>	Time: O(n) Space: O(n) Vi sao? Ta duyệt một lần và dùng HashMap để lưu tổng prefix.
20 Longest Consecutive Sequence Tìm độ dài dãy số liên tiếp dài nhất. Đầu vào: [100,4,200,1,3,2] Đầu ra: 4 (1,2,3,4)	☒ Dùng HashSet ☒ Chỉ bắt đầu từ điểm đầu dãy ☒ Mở rộng & đếm	<pre>public int longestConsecutive(int[] nums) { Set<Integer> set = new HashSet<>(); for (int n : nums) set.add(n); int longest = 0; for (int n : set) { if (!set.contains(n - 1)) { // start of seq int curr = n, count = 1; while (set.contains(curr + 1)) { curr++; count++; } longest = Math.max(longest, count); } } return longest; }</pre>	Time: O(n) Space: O(n) Vi sao? Mỗi số được xét tối đa hai lần.
21 3Sum Tìm tất cả bộ ba giá trị duy nhất trong mảng có tổng bằng 0. Đầu vào: [-1,0,1,2,-1,-4] Đầu ra: [[-1,-1,2], [-1,0,1]]	1. Sắp xếp mảng 2. Cố định một phần tử 3. Two pointers cho hai phần tử còn lại 4. Bỏ qua giá trị trùng	<pre>public List<List<Integer>> threeSum(int[] nums) { Arrays.sort(nums); List<List<Integer>> res = new ArrayList<>(); for (int i = 0; i < nums.length - 2; i++) { if (i > 0 && nums[i] == nums[i - 1]) continue; int l = i + 1, r = nums.length - 1; while (l < r) { int sum = nums[i] + nums[l] + nums[r]; if (sum == 0) { res.add(Arrays.asList(nums[i], nums[l], nums[r])); while (l < r && nums[l] == nums[l + 1]) l++; while (l < r && nums[r] == nums[r - 1]) r--; l++; r--; } else if (sum < 0) l++; else r--; } } return res; }</pre>	Time: O(n³) Space: O(1) Vi sao? Cố định + two pointer → O(n ²) thời gian, O(1) bộ nhớ phụ.
22 Trapping Rain Water Tính lượng nước mưa có thể chứa được. Đầu vào: [0,1,0,2,1,0,1,3,2,1,2,1] Đầu ra: 6	☒ Two pointers ☒ Lưu leftMax & rightMax ☒ Di chuyển con trỏ ở phía thấp hơn	<pre>public int trap(int[] height) { int l = 0, r = height.length - 1; int leftMax = 0, rightMax = 0, water = 0; while (l < r) { if (height[l] < height[r]) { if (height[l] >= leftMax) leftMax = height[l]; else water += leftMax - height[l]; l++; } else { if (height[r] >= rightMax) rightMax = height[r]; else water += rightMax - height[r]; r--; } } return water; }</pre>	Time: O(n) Space: O(1) Vi sao? Duyệt một lần với hai con trỏ.
23 Container With Most Water Tìm hai đường mà cùng trục x tạo thành container chứa được nhiều nước nhất. Đầu vào: [1,8,6,2,5,4,8,3,7] Đầu ra: 49	☒ Two pointers ☒ Tính diện tích ☒ Di chuyển con trỏ ở phía thấp hơn	<pre>public int maxArea(int[] height) { int l = 0, r = height.length - 1, max = 0; while (l < r) { int h = Math.min(height[l], height[r]); max = Math.max(max, h * (r - l)); if (height[l] < height[r]) l++; else r--; } return max; }</pre>	Time: O(n) Space: O(1) Vi sao? Ta thử mọi cặp một cách thông minh bằng hai con trỏ.
24 First Negative Number in Every Window (K) Tìm số âm đầu tiên trong mỗi window kích thước K. Đầu vào: [12,-1,-7,8,-15,30,16,28], K=3 Đầu ra: [-1,-1,-7,-15,-15,0]	☒ Dùng Queue (Deque) ☒ Lưu chỉ số các số âm ☒ Loại các chỉ số ngoài window	<pre>public int[] firstNegative(int[] nums, int k) { Queue<Integer> q = new LinkedList<>(); int[] res = new int[nums.length - k + 1]; int idx = 0; for (int i = 0; i < nums.length; i++) { if (nums[i] < 0) q.offer(i); if (i >= k - 1) { while (!q.isEmpty() && q.peek() <= i - k) q.poll(); res[idx++] = q.isEmpty() ? 0 : nums[q.peek()]; } } return res; }</pre>	Time: O(n) Space: O(k) Vi sao? Mỗi chỉ số được thêm và loại tối đa một lần.



★ QUICK 2 KNOWLEDGE ★

CẨM NANG TẤT CẢ CÁC PATTERN DSA

Những Điều Bạn Cần Biết Trước Khi Phỏng Vấn

Trang: 6

Ngày: _/_/_



Pattern / Problem	Example	Code (Java)	Time & Space
25 Merge K Sorted Lists (Heap) Gộp k linked list đã sắp xếp thành một list đã sắp xếp. Đầu vào: [[1,4,5], [1,3,4],[2,6]] Đầu ra: [1,1,2,3,4,4,5,6]	☒ Dùng Min Heap (PriorityQueue) ☒ Lưu node đầu của mỗi list ☒ Lấy min & thêm node kế tiếp	<pre>class ListNode { int val; ListNode next; ListNode(int x){val=x;} } public ListNode mergeKLists(ListNode[] lists) { PriorityQueue<ListNode> pq = new PriorityQueue<>((a,b) -> a.val - b.val); for (ListNode node : lists) if (node != null) pq.offer(node); ListNode dummy = new ListNode(0), curr = dummy; while (!pq.isEmpty()) { ListNode node = pq.poll(); curr.next = node; curr = curr.next; if (node.next != null) pq.offer(node.next); } return dummy.next; }</pre>	Time: $O(N \log k)$ Space: $O(k)$ Vi sao? Mỗi node được push và pop khỏi heap.
26 Detect Cycle in Linked List Phát hiện linked list có chứa chu trình không. Đầu vào: 1 → 2 → 3 → 4 → 2 (Output: true)	☒ Dùng Fast & Slow Pointers ☒ Fast đi chuyển 2x, Slow đi chuyển 1x ☒ Nếu gặp nhau, chu trình tồn tại	<pre>public boolean hasCycle(ListNode head) { if (head == null head.next == null) return false; ListNode slow = head, fast = head.next; while (fast != null && fast.next != null) { if (slow == fast) return true; slow = slow.next; fast = fast.next.next; } return false; }</pre>	Time: $O(n)$ Space: $O(1)$ Vi sao? Con trỏ nhanh sẽ gặp con trỏ chậm bên trong chu trình.
27 LRU Cache (Design) Thiết kế cache loại bỏ phần tử ít dùng gần đây nhất khi đầy. Operations: get(key), put(key, value)	☒ Dùng HashMap (key → Node) ☒ Dùng Doubly Linked List ☒ $O(1)$ cho get/put	<pre>class LRUCache { int cap; Map<Integer, Node> map = new HashMap<>(); Node head = new Node(0,0), tail = new Node(0,0); public LRUCache(int capacity) { cap = capacity; head.next = tail; tail.prev = head; } // get & put operations handle in O(1) using DLL + Map }</pre>	Time: $O(1)$ (cho get & put) Space: $O(capacity)$ Vi sao? Ta dùng DLL + HashMap để đạt thời gian hằng số.
28 Topological Sort (Graph) Tìm một thứ tự hợp lệ của các node trong DAG. Đầu vào: Edges = [[5,2],[5,0],[4,0],[4,1],[2,3],[3,1]] Đầu ra: [5,4,2,3,1,0]	☒ Tính indegree ☒ Dùng Queue (BFS) ☒ Loại node có indegree = 0	<pre>public int[] topoSort(int V, int[][] edges) { List<List<Integer>> adj = new ArrayList<>(); for (int i=0;i<V;i++) adj.add(new ArrayList<>()); int[] indegree = new int[V]; for (int[] e : edges) { adj.get(e[0]).add(e[1]); indegree[e[1]]++; } Queue<Integer> q = new LinkedList<>(); for (int i=0;i<V;i++) if (indegree[i]==0) q.offer(i); int[] res = new int[V]; int idx = 0; while (!q.isEmpty()) { int node = q.poll(); res[idx++] = node; for (int nei : adj.get(node)) if (--indegree[nei] == 0) q.offer(nei); } return res; }</pre>	Time: $O(V+E)$ Space: $O(V+E)$ Vi sao? Ta duyệt mỗi node và mỗi cạnh một lần.
29 Dijkstra's Algorithm (Shortest Path) Tìm đường đi ngắn nhất từ nguồn đến mọi node. Đầu vào: Graph với trọng số dương Đầu ra: Khoảng cách ngắn nhất tới mọi node	☒ Dùng Min Heap ☒ Relax tất cả cạnh kề ☒ Cập nhật khoảng cách nếu nhỏ hơn	<pre>public int[] dijkstra(int V, List<List<int[]>> adj, int src) { int[] dist = new int[V]; Arrays.fill(dist, Integer.MAX_VALUE); PriorityQueue<int[]> pq = new PriorityQueue<>((a,b)->a[0]-b[0]); dist[src] = 0; pq.offer(new int[]{0, src}); while (!pq.isEmpty()) { int[] cur = pq.poll(); int d = cur[0], u = cur[1]; for (int[] nei : adj.get(u)) { int v = nei[0], wt = nei[1]; if (d + wt < dist[v]) { dist[v] = d + wt; pq.offer(new int[]{dist[v], v}); } } } return dist; }</pre>	Time: $O((V+E) \log V)$ Space: $O(V)$ Vi sao? Heap tốn log V. Ta relax mỗi cạnh một lần.
30 Trie (Prefix Tree) Lưu trữ chuỗi hiệu quả và tìm kiếm theo prefix. Operations: insert(word), search(word), startsWith(prefix)	☒ Mỗi node có 26 con ☒ Tìm kiếm prefix nhanh trong $O(L)$ ☒ Hữu ích trong Auto-Complete, Dict, v.v.	<pre>class TrieNode { TrieNode[] child = new TrieNode[26]; boolean isEnd; } class Trie { TrieNode root = new TrieNode(); public void insert(String word) { TrieNode node = root; for (char c : word.toCharArray()) { int i=c-'a'; if (node.child[i]==null) node.child[i]=new TrieNode(); node = node.child[i]; } node.isEnd = true; } public boolean search(String word) { TrieNode node = root; for (char c : word.toCharArray()) { int i=c-'a'; if (node.child[i]==null) return false; node = node.child[i]; } return node.isEnd; } }</pre>	Time: $O(L)$ Space: $O(N*L)$ Vi sao? L = độ dài từ. N = số từ trong Trie.



★ QUICK 2 KNOWLEDGE ★

CẨM NANG TẤT CẢ CÁC PATTERN DSA

Những Điều Bạn Cần Biết Trước Khi Phỏng Vấn

Trang: 7

Ngày: __/__/__



Pattern / Problem	Example	Code (Java)	Time & Space
31 Evaluate Reverse Polish Notation (RPN) Tính giá trị của biểu thức số học ở dạng Reverse Polish Notation. Đầu vào: ["2","1","+","3","*"] Đầu ra: 9	☒ Dùng Stack ☒ Push toán hạng ☒ Áp dụng toán tử lên hai phần tử trên đỉnh	<pre>public int evalRPN(String[] tokens) { Stack<Integer> st = new Stack<>(); for (String t : tokens) { if (t.equals("+") t.equals("-") t.equals("*") t.equals("/") { int b = st.pop(), a = st.pop(); if (t.equals("+")) st.push(a + b); else if (t.equals("-")) st.push(a - b); else if (t.equals("*")) st.push(a * b); else st.push(a / b); } else { st.push(Integer.parseInt(t)); } } return st.pop(); }</pre>	Time: O(n) Space: O(n) Vì sao? Mỗi token được xử lý một lần. Dùng Stack.
32 Next Greater Element I Tìm phần tử lớn hơn kế tiếp của mỗi phần tử trong nums1, trong nums2. Đầu vào: nums1 = [4,1,2], nums2 = [1,3,4,2] Đầu ra: [-1,3,-1]	☒ Dùng Monotonic Stack ☒ Lưu phần tử theo thứ tự giảm dần ☒ Map next greater	<pre>public int[] nextGreaterElement(int[] nums1, int[] nums2) { Map<Integer, Integer> map = new HashMap<>(); Stack<Integer> st = new Stack<>(); for (int num : nums2) { while (!st.isEmpty() && st.peek() < num) { map.put(st.pop(), num); } st.push(num); } while (!st.isEmpty()) map.put(st.pop(), -1); int[] res = new int[nums1.length]; for (int i = 0; i < nums1.length; i++) res[i] = map.get(nums1[i]); return res; }</pre>	Time: O(n) Space: O(n) Vì sao? Mỗi phần tử được push và pop tối đa một lần.
33 Sort Colors (Dutch National Flag) Sắp xếp mảng chứa 0s, 1s, và 2s. Đầu vào: [2,0,2,1,1,0] Đầu ra: [0,0,1,1,2,2]	☒ Ba con trỏ: low, mid, high ☒ Partition trong một lần duyệt	<pre>public void sortColors(int[] nums) { int low = 0, mid = 0, high = nums.length - 1; while (mid <= high) { if (nums[mid] == 0) swap(nums, low++, mid++); else if (nums[mid] == 1) mid++; else swap(nums, mid, high--); } } private void swap(int[] a, int i, int j) { int t = a[i]; a[i] = a[j]; a[j] = t; }</pre>	Time: O(n) Space: O(1) Vì sao? Mỗi phần tử được xử lý một lần.
34 Merge k Sorted Lists Gộp k linked list đã sắp xếp và trả về một list đã sắp xếp. Đầu vào: k lists Đầu ra: List đã gộp	☒ Dùng Min Heap (PriorityQueue) ☒ Luôn lấy node nhỏ nhất ☒ Thêm node kế tiếp từ cùng list	<pre>class ListNode { int val; ListNode next; ListNode(int v){ val=v; } } public ListNode mergeKLists(ListNode[] lists) { PriorityQueue<ListNode> pq = new PriorityQueue<>((a,b) -> a.val - b.val); for (ListNode node : lists) if (node != null) pq.offer(node); ListNode dummy = new ListNode(0), cur = dummy; while (!pq.isEmpty()) { ListNode node = pq.poll(); cur.next = node; cur = cur.next; if (node.next != null) pq.offer(node.next); } return dummy.next; }</pre>	Time: O(n log k) (n = tổng số node) Space: O(k) Vì sao? Mỗi lần chèn/xóa node khỏi heap tốn log k.
35 LRU Cache Thiết kế cấu trúc dữ liệu theo LRU Cache. Operations: get(key), put(key, value) Tất cả đều O(1).	☒ HashMap cho key → node ☒ Doubly Linked List để duy trì thứ tự ☒ Di chuyển phần tử vừa dùng lên đầu	<pre>class LRUCache { class Node { int key, val; Node prev, next; Node(int k, int v){key=k;val=v;} } private int cap; private Map<Integer, Node> map = new HashMap<>(); private Node head = new Node(0,0), tail = new Node(0,0); public LRUCache(int capacity) { cap = capacity; head.next = tail; tail.prev = head; } // get() và put() operations dùng DLL + Map }</pre>	Time: O(1) Space: O(capacity) Vì sao? Tất cả thao tác dùng DLL + HashMap.
36 Find Duplicate Number Tìm số bị trùng trong mảng chứa n+1 số nguyên, mỗi số nằm trong [1,n]. (Không sửa mảng gốc) Đầu vào: [1,3,4,2,2] Đầu ra: 2	☒ Floyd's Cycle Detection ☒ Xem mảng như linked list ☒ Tìm điểm bắt đầu chu trình	<pre>public int findDuplicate(int[] nums) { int slow = nums[0], fast = nums[0]; do { slow = nums[slow]; fast = nums[nums[fast]]; } while (slow != fast); slow = nums[0]; while (slow != fast) { slow = nums[slow]; fast = nums[fast]; } return slow; }</pre>	Time: O(n) Space: O(1) Vì sao? Cách tiếp cận dựa trên chu trình của linked list.



Pattern / Problem	Example	Code (Java)	Time & Space
37 Valid Anagram Kiểm tra hai chuỗi có phải anagram của nhau không. Đầu vào: s = "listen", t = "silent" Đầu ra: true	☒ So sánh tần suất ký tự ☒ Dùng HashMap / int[26]	<pre>public boolean isAnagram(String s, String t) { if (s.length() != t.length()) return false; int[] count = new int[26]; for (char c : s.toCharArray()) count[c - 'a']++; for (char c : t.toCharArray()) count[c - 'a']--; for (int x : count) if (x != 0) return false; return true; }</pre>	Time: O(n) Space: O(1) Vì sao? Ta dùng mảng cố định kích thước 26.
38 Topological Sort (Indegree BFS) Trả về thứ tự topo của DAG. Đầu vào: V = 4, Edges = [[0,1], [0,2], [1,2], [2,3]] Đầu ra: [0,1,2,3] hoặc [0,2,1,3]	☒ Tính indegree của tất cả node ☒ Thêm node có indegree 0 vào queue ☒ BFS & giảm indegree	<pre>public int[] topoSort(int V, int[][] edges) { List<List<Integer>> adj = new ArrayList<>(); for (int i = 0; i < V; i++) adj.add(new ArrayList<>()); int[] indeg = new int[V]; for (int[] e : edges) { adj.get(e[0]).add(e[1]); indeg[e[1]]++; } Queue<Integer> q = new LinkedList<>(); for (int i = 0; i < V; i++) if (indeg[i] == 0) q.offer(i); int[] res = new int[V]; int idx = 0; while (!q.isEmpty()) { int node = q.poll(); res[idx++] = node; for (int neigh : adj.get(node)) if (--indeg[neigh] == 0) q.offer(neigh); } return (idx == V) ? res : new int[0]; // cycle if not all visited }</pre>	Time: O(V + E) Space: O(V + E) Vì sao? Mỗi node và cạnh được xử lý một lần.
39 Detect Cycle in Directed Graph (DFS) Kiểm tra đồ thị có hướng có chứa chu trình không. Đầu vào: V = 3, Edges = [[0,1], [1,2], [2,0]] Đầu ra: true	☒ DFS với 3 trạng thái: 0 = chưa thăm, 1 = đang thăm, 2 = đã thăm ☒ Phát hiện back edge khi gặp lại node đang thăm (state 1)	<pre>class Solution { public boolean isCyclic(int V, List<List<Integer>> adj) { int[] state = new int[V]; // 0,1,2 for (int i = 0; i < V; i++) if (state[i] == 0 && dfs(i, adj, state)) return true; return false; } private boolean dfs(int node, List<List<Integer>> adj, int[] state) { state[node] = 1; // visiting for (int nei : adj.get(node)) { if (state[nei] == 1) return true; // back edge if (state[nei] == 0 && dfs(nei, adj, state)) return true; } state[node] = 2; // visited return false; } }</pre>	Time: O(V + E) Space: O(V) Vì sao? Mỗi node và cạnh được thăm tối đa một lần.
40 Dijkstra's Algorithm (Shortest Path) Tìm đường đi ngắn nhất từ nguồn đến mọi node trong đồ thị có trọng số không âm. Đầu vào: V=5, src=0 Đầu ra: Mảng khoảng cách	☒ Dùng Min Heap (PriorityQueue) ☒ Relax các cạnh ☒ Cập nhật khoảng cách	<pre>public int[] dijkstra(int V, List<List<int[]>> adj, int src) { int[] dist = new int[V]; Arrays.fill(dist, Integer.MAX_VALUE); PriorityQueue<int[]> pq = new PriorityQueue<>(); Comparator.comparingInt(a -> a[1]); pq.offer(new int[]{src, 0}); // {node, dist} while (!pq.isEmpty()) { int[] cur = pq.poll(); int u = cur[0], d = cur[1]; if (d > dist[u]) continue; for (int[] nbr : adj.get(u)) { int v = nbr[0], w = nbr[1]; if (d + w < dist[v]) { dist[v] = d + w; pq.offer(new int[]{v, dist[v]}); } } } return dist; }</pre>	Time: O((V + E) log V) Space: O(V + E) Vì sao? Mỗi node được push vào heap một lần và relax một cạnh.
41 Word Ladder (Length) Tìm độ dài chuỗi biến đổi ngắn nhất từ beginWord đến endWord. Đầu vào: begin = "hit", end = "cog", wordList = ["hot", "dot", "dog", "lot", "log", "cog"] Đầu ra: 5	☒ BFS theo từng level ☒ Sinh tất cả biến đổi một ký tự ☒ Dùng HashSet để tra cứu nhanh	<pre>public int ladderLength(String begin, String end, List<String> wordList) { Set<String> dict = new HashSet<>(wordList); if (!dict.contains(end)) return 0; Queue<String> q = new LinkedList<>(); q.offer(begin); Set<String> vis = new HashSet<>(); vis.add(begin); int level = 1; while (!q.isEmpty()) { int size = q.size(); while (size-- > 0) { String word = q.poll(); if (word.equals(end)) return level; char[] arr = word.toCharArray(); for (int i = 0; i < arr.length; i++) { char old = arr[i]; for (char c = 'a'; c <= 'z'; c++) { arr[i] = c; String next = new String(arr); if (dict.contains(next) && !vis.add(next)) q.offer(next); } arr[i] = old; } level++; } return 0; } }</pre>	Time: O(N * M * 26) (N = số từ, M = độ dài) Space: O(N * M) Vì sao? Với mỗi từ ta sinh M*26 từ láng giềng.
42 Serialize & Deserialize Binary Tree Chuyển binary tree thành chuỗi và khôi phục lại. Dùng level order traversal.	☒ Dùng queue cho level order ☒ Dùng "w" cho node null ☒ Khôi phục bằng queue	<pre>// Serialize public String serialize(TreeNode root) { if (root == null) return "#"; StringBuilder sb = new StringBuilder(); Queue<TreeNode> q = new LinkedList<>(); q.offer(root); while (!q.isEmpty()) { TreeNode node = q.poll(); if (node == null) { sb.append("#,"); } else { sb.append(node.val).append((",",)); } q.offer(node.left); q.offer(node.right); } return sb.toString(); } // Deserialize (dùng lại queue để khôi phục theo level order)</pre>	Time: O(N) Space: O(N) Vì sao? Mỗi node được thăm một lần. N = số node.



Pattern / Problem	Example	Code (Java)	Time & Space
43 Implement Trie (Prefix Tree) Cài đặt insert, search và startsWith cho Trie. Đầu vào: ["insert","search","search","starts","starts","starts"] ["apple","apple","apple","apple"] Đầu ra: [null, true, false, true]	☐ Node có 26 con ☐ Chèn rừng ký tự ☐ Search & startsWith duyệt cây	<pre>class TrieNode { TrieNode[] children = new TrieNode[26]; boolean isEnd; } class Trie { TrieNode root = new TrieNode(); public void insert(String word) { TrieNode node = root; for (char c : word.toCharArray()) { int idx = c - 'a'; if (node.children[idx] == null) node.children[idx] = new TrieNode(); node = node.children[idx]; } node.isEnd = true; } public boolean search(String word) { TrieNode node = searchNode(word); return node != null && node.isEnd; } public boolean startsWith(String prefix) { return searchNode(prefix) != null; } private TrieNode searchNode(String key) { TrieNode node = root; for (char c : key.toCharArray()) { int idx = c - 'a'; if (node.children[idx] == null) return null; node = node.children[idx]; } return node; } }</pre>	Time: O(L) n = độ dài từ Space: O(26 * L) n = tổng số nodes Y/N? Tuyệt vời. Mỗi node có 26 con trỏ.
44 LRU Cache (Least Recently Used) Thiết kế cấu trúc dữ liệu theo LRU Cache. Đầu vào: capacity = 2, put(1,1), put(2,2), get(1), put(3,3), get(2) Đầu ra: 1, -1	☐ Dùng Doubly Linked List để giữ thứ tự ☐ Dùng HashMap để truy cập nhanh ☐ Dùng doubly node vừa đóng lại đầu ☐ Xóa LRU khi chỗi	<pre>class LRUCache { class Node { int key, val; Node prev, next; Node(int k, int v) { key=k; val=v; } } int cap; Map<Integer, Node> map = new HashMap(); Node head = new Node(0,0), tail = new Node(0,0); public LRUCache(int capacity) { cap = capacity; head.next = tail; tail.prev = head; } public int get(int key) { if (!map.containsKey(key)) return -1; Node node = map.get(key); node.val = value; remove(node); addFront(node); return node.val; } public void put(int key, int value) { if (!map.containsKey(key)) { Node node = new Node(key, value); map.put(key, node); addFront(node); if (map.size() == cap) { Node lru = tail.prev; remove(lru); map.remove(lru.key); } } private void addFront(Node node) { node.next = head.next; node.prev = head; head.next.prev = node; head.next = node; } private void remove(Node node) { node.prev.next = node.next; node.next.prev = node.prev; } } }</pre>	Time: O(1) Space: O(capacity) Y/N? Tuyệt vời. Dùng doubly node để truy cập nhanh. Dùng HashMap để lưu trữ.
45 Median of Two Sorted Arrays Tìm trung vị của hai mảng đã sắp xếp. Đầu vào: [1,3], [2] Đầu ra: 2.0	☐ Binary Search trên mảng nhỏ hơn ☐ Partition cả hai mảng ☐ Dùng maxLeft & minRight	<pre>public double findMedianSortedArrays(int[] a, int[] b) { if (a.length > b.length) return findMedianSortedArrays(b, a); int n = a.length + b.length; int low = 0, high = n; while (low < high) { int cutA = (low + high) / 2; int cutB = (n - cutA) / 2; int lA = (cutA-1) > 0 ? a[cutA-1] : Integer.MIN_VALUE; int rA = (cutA) < a.length ? a[cutA] : Integer.MAX_VALUE; int lB = (cutB-1) > 0 ? b[cutB-1] : Integer.MIN_VALUE; int rB = (cutB) < b.length ? b[cutB] : Integer.MAX_VALUE; if (lA <= rB && lB <= rA) { if ((n % 2 == 1) && (n % 2 == 1)) return Math.max(lA, lB); return (Math.max(lA, lB) + Math.min(rA, rB)) / 2.0; } else if (lA > rB) high = cutA - 1; else low = cutA + 1; } return 0.0; }</pre>	Time: O(log(min(m,n))) Space: O(1) Y/N? Binary search giảm không gian tìm kiếm trên mảng nhỏ hơn.
46 Maximal Rectangle in Binary Matrix Tìm hình chữ nhật lớn nhất chỉ chứa 1s. Đầu vào: ["1","0","1","1","0"], ["1","0","1","1","0"], ["1","1","1","1","1"], ["1","0","0","1","1"], ["1","0","0","1","0"] Đầu ra: 6	☐ Cài mỗi hàng là histogram ☐ Tính chiều cao ☐ Dùng Largest Rectangle in Histogram	<pre>public int maximalRectangle(char[][] matrix) { if (matrix.length == 0) return 0; int rows = matrix.length, cols = matrix[0].length, max = 0; int[] height = new int[cols]; for (int i = 0; i < rows; i++) { for (int j = 0; j < cols; j++) { height[j] = (matrix[i][j] == '1') ? height[j] + 1 : 0; } max = Math.max(max, largestRectangleArea(height)); } return max; } int largestRectangleArea(int[] height) { Stack<Integer> stack = new Stack(); stack.push(-1); for (int i = 0; i < height.length; i++) { while (stack.peek() != -1 && height[i] <= height[stack.peek()]) { stack.pop(); } stack.push(i); } while (stack.peek() != -1) { stack.pop(); } return 0; }</pre>	Time: O(rows * cols) Space: O(cols) Y/N? Mỗi hàng xử lý một lần, histogram tính trong O(cols) mỗi hàng.
47 Serialize & Deserialize N-ary Tree Chuyển N-ary tree thành chuỗi và khôi phục lại. ☐ Dùng preorder ☐ Lưu số lượng con ☐ Dùng queue để khôi phục	☐ Dùng preorder ☐ Lưu số lượng con ☐ Dùng queue để khôi phục	<pre>// Serialize public String serialize(Node root) { if (root == null) return ""; StringBuilder sb = new StringBuilder(); Queue<Node> q = new LinkedList(); q.offer(root); while (!q.isEmpty()) { Node node = q.poll(); sb.append(node.val).append(" "); for (Node child : node.children) q.offer(child); } return sb.toString(); } // Deserialize public Node deserialize(String data) { if (data == null data.isEmpty()) return null; String[] nodes = data.split(" "); Queue<Node> q = new LinkedList(); q.offer(new Node(nodes[0])); int i = 1; while (i < nodes.length) { Node node = q.poll(); for (Node child : node.children) q.offer(child); } return q.poll(); }</pre>	Time: O(N) Space: O(N) Y/N? Mỗi node được thêm một lần khi serialize và deserialize.
48 Sliding Window Maximum Tìm giá trị lớn nhất trong mỗi window kích thước k. Đầu vào: nums=[1,3,-1,-3,5,6,7], k=3 Đầu ra: [3,5,5,6,7]	☐ Dùng Deque (chỉ số) ☐ Loại chỉ số ngoài window ☐ Dùng vị trí thứ tự giảm dần	<pre>public int[] maxSlidingWindow(int[] nums, int k) { int n = nums.length; int[] res = new int[n - k + 1]; Deque<Integer> dq = new ArrayDeque(); for (int i = 0; i < n; i++) { while (!dq.isEmpty() && nums[i] >= nums[dq.peekLast()]) { dq.pollLast(); } dq.offer(i); if (i - dq.peekFirst() >= k) { res[i - k] = nums[dq.peekFirst()]; dq.pollFirst(); } } return res; }</pre>	Time: O(n) Space: O(k) Y/N? Mỗi chỉ số được thêm và loại khỏi da một lần khi deque.



★ QUICK 2 KNOWLEDGE ★

CẨM NANG TẤT CẢ CÁC PATTERN DSA

Những Điều Bạn Cần Biết Trước Khi Phỏng Vấn

Trang: 10

Ngày: _/_/_



Pattern / Problem	Example	Code (Java)	Time & Space
49 Find Subarray with Given Sum (K) Tìm một subarray có tổng các phần tử bằng k. Đầu vào: [1,2,3,7,5], k=12 Đầu ra: [2,3,7]	☒ Dùng Prefix Sum + HashMap ☒ Lưu prefixSum → chỉ số đầu tiên ☒ Kiểm tra (prefixSum - k) có tồn tại	<pre>public int[] subarraySum(int[] nums, int k) { Map<Integer, Integer> map = new HashMap<>(); int sum = 0; for (int i = 0; i < nums.length; i++) { sum += nums[i]; if (map.containsKey(sum - k)) { return Arrays.copyOfRange(nums, map.get(sum - k) + 1, i + 1); } map.putIfAbsent(sum, i); } return new int[0]; }</pre>	Time: O(n) Space: O(n) Vi sao? Ta duyệt một lần và dùng HashMap để lưu tổng prefix.
50 Longest Substring Without Repeating Characters Tìm độ dài substring dài nhất không có ký tự lặp lại. Đầu vào: s = "abcabcbb" Đầu ra: 3 ("abc")	☒ Dùng Sliding Window + HashSet ☒ Mở rộng phải, thu hẹp trái khi trùng ☒ Theo dõi độ dài lớn nhất	<pre>public int lengthOfLongestSubstring(String s) { Set<Character> set = new HashSet<>(); int l = 0, max = 0; for (int r = 0; r < s.length(); r++) { while (set.contains(s.charAt(r))) { set.remove(s.charAt(l++)); } set.add(s.charAt(r)); max = Math.max(max, r - l + 1); } return max; }</pre>	Time: O(n) Space: O(k) (k = số ký tự) Vi sao? Mỗi ký tự được thêm và loại tối đa một lần.
51 Longest Repeating Character Replacement Thay tối đa k ký tự để có substring lặp lại dài nhất. Đầu vào: s = "AABABBA", k = 1 Đầu ra: 4	☒ Sliding Window ☒ Lưu tần suất ký tự ☒ Giữ tần suất lớn nhất trong window	<pre>public int characterReplacement(String s, int k) { int[] freq = new int[26]; int l = 0, maxCount = 0, maxLen = 0; for (int r = 0; r < s.length(); r++) { int idx = s.charAt(r) - 'A'; freq[idx]++; maxCount = Math.max(maxCount, freq[idx]); while ((r - l + 1) - maxCount > k) { freq[s.charAt(l) - 'A']--; l++; } maxLen = Math.max(maxLen, r - l + 1); } return maxLen; }</pre>	Time: O(n) Space: O(26) = O(1) Vi sao? Window mở rộng và thu hẹp mỗi lần một lần.
52 Minimum Window Substring Tìm window nhỏ nhất trong s chứa tất cả ký tự của t. Đầu vào: s = "ADOBECODEBANC", t = "ABC" Đầu ra: "BANC"	☒ Sliding Window ☒ Dùng bảng tần suất cho t ☒ Mở rộng window & thu hẹp để tối ưu	<pre>public String minWindow(String s, String t) { if (s.length() < t.length()) return ""; int[] need = new int[128]; for (char c : t.toCharArray()) need[c]++; int l = 0, count = t.length(), minLen = Integer.MAX_VALUE; int start = 0; for (int r = 0; r < s.length(); r++) { if (need[s.charAt(r)]-- > 0) count--; while (count == 0) { if (r - l + 1 < minLen) { minLen = r - l + 1; start = l; } if (need[s.charAt(l++)]++ == 0) count++; } } return minLen == Integer.MAX_VALUE ? "" : s.substring(start, start + minLen); }</pre>	Time: O(n) Space: O(128) = O(1) Vi sao? Mỗi ký tự được thêm và loại tối đa một lần.
53 Sliding Window Maximum Tìm phần tử lớn nhất trong mỗi sliding window kích thước k. Đầu vào: [1,3,-1,-3,5,3,6,7], k = 3 Đầu ra: [3,3,5,5,6,7]	☒ Dùng Deque (chỉ số) ☒ Loại chỉ số ngoài window ☒ Duy trì thứ tự giảm dần trong deque	<pre>public int[] maxSlidingWindow(int[] nums, int k) { Deque<Integer> dq = new ArrayDeque<>(); int[] res = new int[nums.length - k + 1]; int idx = 0; for (int i = 0; i < nums.length; i++) { while (!dq.isEmpty() && dq.peekFirst() <= i - k) dq.pollFirst(); while (!dq.isEmpty() && nums[dq.peekLast()] <= nums[i]) dq.pollLast(); dq.offerLast(i); if (i >= k - 1) res[idx++] = nums[dq.peekFirst()]; } return res; }</pre>	Time: O(n) Space: O(k) Vi sao? Mỗi chỉ số được thêm và loại tối đa một lần khỏi deque.
54 Trapping Rain Water Cho n số không âm, tính tổng lượng nước mưa có thể chứa được. Đầu vào: [0,1,0,2,1,0,1,3,2,1,2,1] Đầu ra: 6	☒ Two Pointers ☒ Lưu leftMax & rightMax ☒ Di chuyển con trỏ ở phía thấp hơn	<pre>public int trap(int[] height) { int l = 0, r = height.length - 1; int leftMax = 0, rightMax = 0, water = 0; while (l < r) { if (height[l] < height[r]) { leftMax = Math.max(leftMax, height[l]); water += leftMax - height[l]; l++; } else { rightMax = Math.max(rightMax, height[r]); water += rightMax - height[r]; r--; } } return water; }</pre>	Time: O(n) Space: O(1) Vi sao? Two pointers đảm bảo duyệt một lần với O(1) bộ nhớ phụ.



★ QUICK 2 KNOWLEDGE ★

CẨM NANG TẤT CẢ CÁC PATTERN DSA

Những Điều Bạn Cần Biết Trước Khi Phỏng Vấn

Trang: 11

Ngày: __/__/__



Pattern / Problem	Example	Code (Java)	Time & Space
55 Container With Most Water Tìm hai đường mà cùng trục x tạo thành container chứa được nhiều nước nhất. Đầu vào: [1,8,6,2,5,4,8,3,7] Đầu ra: 49	- Two Pointers - Di chuyển con trỏ ở phía thấp hơn - Tính diện tích & theo dõi max	<pre>public int maxArea(int[] height) { int l = 0, r = height.length - 1, max = 0; while (l < r) { int h = Math.min(height[l], height[r]); // // giới hạn chiều cao int w = r - l; // chiều rộng max = Math.max(max, h * w); // diện tích if (height[l] < height[r]) l++; else r--; } return max; }</pre>	Time: $O(n)$ Space: $O(1)$ Vì sao? Ta di chuyển con trỏ ở phía thấp hơn vì nó giới hạn diện tích.
56 3 Sum Tìm tất cả bộ ba giá trị duy nhất trong mảng có tổng bằng 0. Đầu vào: [-1,0,1,2,-1,-4] Đầu ra: [[-1,-1,2], [-1,0,1]]	- Sắp xếp mảng - Cố định một số - Two pointers cho hai số còn lại - Bỏ qua giá trị trùng	<pre>public List<List<Integer>> threeSum(int[] nums) { List<List<Integer>> res = new ArrayList<>(); Arrays.sort(nums); for (int i = 0; i < nums.length - 2; i++) { if (i > 0 && nums[i] == nums[i-1]) continue; int l = i + 1, r = nums.length - 1; while (l < r) { int sum = nums[i] + nums[l] + nums[r]; if (sum == 0) { res.add(Arrays.asList(nums[i], nums[l], nums[r])); l++; r--; while (l < r && nums[l] == nums[l-1]) l++; while (l < r && nums[r] == nums[r+1]) r--; } else if (sum < 0) l++; else r--; } } return res; }</pre>	Time: $O(n^2)$ Space: $O(1)$ Vì sao? Sau khi sắp xếp, ta cố định một số rồi dùng two pointers tìm hai số còn lại.
57 Kth Smallest Element in Sorted Matrix Tìm phần tử nhỏ thứ k trong ma trận $n \times n$ đã sắp xếp.	- Binary Search trên khoảng giá trị - Đếm phần tử $\leq$ mid - Điều chỉnh khoảng tìm kiếm	<pre>public int kthSmallest(int[][] matrix, int k) { int n = matrix.length, lo = matrix[0][0], hi = matrix[n-1][n-1], ans = -1; while (lo <= hi) { int mid = lo + (hi - lo) / 2; int count = countLessEqual(matrix, mid); if (count >= k) { ans = mid; hi = mid - 1; } else lo = mid + 1; } return ans; } private int countLessEqual(int[][] mat, int x) { int n = mat.length, r = n - 1, c = 0, count = 0; while (r >= 0 && c < n) { if (mat[r][c] <= x) { count += r + 1; c++; } else r--; } return count; }</pre>	Time: $O(n \log(\max-min))$ Space: $O(1)$ Vì sao? Binary search thu hẹp khoảng giá trị, đếm số $\leq$ mid trong $O(n)$.
58 Number of Islands Cho lưới 2D gồm '1' (đất liền) và '0' (nước), đếm số lượng đảo. Một đảo được bao quanh bởi nước và nối theo chiều ngang hoặc dọc.	- Duyệt DFS/BFS - Đánh dấu đã thăm - Duyệt 4 hướng (Trên, Dưới, Trái, Phải)	<pre>public int numIslands(char[][] grid) { int m = grid.length, n = grid[0].length, count = 0; for (int i = 0; i < m; i++) { for (int j = 0; j < n; j++) { if (grid[i][j] == '1') { count++; dfs(grid, i, j); } } } return count; } private void dfs(char[][] g, int i, int j) { int m = g.length, n = g[0].length; if (i < 0 j < 0 i >= m j >= n g[i][j] != '1') return; g[i][j] = '0'; dfs(g, i+1, j); dfs(g, i-1, j); dfs(g, i, j+1); dfs(g, i, j-1); }</pre>	Time: $O(m * n)$ Space: $O(m * n)$ (trường hợp xấu nhất, đệ quy) Vì sao? Ta thăm mỗi ô tối đa một lần trong DFS và đánh dấu đã thăm.
59 Course Schedule (Cycle in Directed Graph) Xác định có thể hoàn thành tất cả môn học với các điều kiện tiên quyết cho trước hay không.	- Topological Sort (Kahn's Algorithm) - Indegree của các node - BFS trên các node có indegree 0	<pre>public boolean canFinish(int numCourses, int[][] pre) { List<List<Integer>> adj = new ArrayList<>(); for (int i = 0; i < numCourses; i++) adj.add(new ArrayList<>()); int[] indeg = new int[numCourses]; for (int[] p : pre) { adj.get(p[1]).add(p[0]); indeg[p[0]]++; } Queue<Integer> q = new LinkedList<>(); for (int i = 0; i < numCourses; i++) if (indeg[i] == 0) q.offer(i); int visited = 0; while (!q.isEmpty()) { int u = q.poll(); visited++; for (int v : adj.get(u)) if (--indeg[v] == 0) q.offer(v); } return visited == numCourses; }</pre>	Time: $O(V + E)$ Space: $O(V + E)$ Vì sao? Nếu duyệt được hết các môn học theo thứ tự topo thì không có chu trình.
60 Minimum Spanning Tree (Kruskal's Algorithm) Tìm chi phí nhỏ nhất của cây khung nhỏ nhất của đồ thị vô hướng liên thông.	- Sắp xếp các cạnh theo trọng số - Dùng Union-Find - Chọn cạnh không tạo chu trình	<pre>public int kruskal(int V, int[][] edges) { Arrays.sort(edges, Comparator.comparingInt(a -> a[2])); UnionFind uf = new UnionFind(V); int cost = 0, cnt = 0; for (int[] e : edges) { int u = e[0], v = e[1], w = e[2]; if (uf.union(u, v)) { cost += w; cnt++; if (cnt == V - 1) break; } } return cost; }</pre>	Time: $O(E \log E)$ Space: $O(V)$ Vì sao? Ta luôn chọn cạnh nhỏ nhất mà không tạo chu trình.



Pattern / Problem	Example	Code (Java)	Time & Space
61 Alien Dictionary (Topological Sort) Tìm thứ tự các ký tự trong ngôn ngữ dựa trên từ điển đã sắp xếp. Đầu vào: ["bat","tab","abc","cab"] Đầu ra: "bac"	❑ Xây đồ thị từ các từ để tìm. ❑ Topological Sort (Kahn's Algo) ❑ Nếu có chu trình, trả về -1	<pre>public String alienOrder(String[] words) { Map<Character, List<Character>> graph = new HashMap<>(); int[] indeg = new int[26]; for (String w : words) for (char c : w.toCharArray()) graph.putIfAbsent(c, new ArrayList<>()); for (int i = 0; i < words.length - 1; i++) { String w1 = words[i], w2 = words[i+1]; int len = Math.min(w1.length(), w2.length()); for (int j = 0; j < len; j++) { char c1 = w1.charAt(j), c2 = w2.charAt(j); if (c1 < c2) { if (!graph.get(c1).contains(c2)) { graph.get(c1).add(c2); indeg[c2 - 'a']++; } break; } } if (w1.length() == w2.length() && w1.charAt(len) != w2.charAt(len)) return ""; } Queue<Character> q = new LinkedList<>(); StringBuilder sb = new StringBuilder(); for (char c : graph.keySet()) if (indeg[c - 'a'] == 0) q.offer(c); while (!q.isEmpty()) { char c = q.poll(); sb.append(c); for (char nei : graph.get(c)) if (--indeg[nei - 'a'] == 0) q.offer(nei); } return sb.length() == graph.size() ? sb.toString() : ""; }</pre>	Time: $O(V + E)$ $(V = 26, E = \sum \text{Space: } O(V + E))$ Yikes! Topo sort các ký tự. Nếu có chu trình, không thể sắp xếp.
62 Accounts Merge (Disjoint Set Union) Cấp các tài khoản có cùng email. Đầu vào: ["John","john@gmail.com","john@mail.com"], ["John","john@gmail.com","john@mail.com"], ["John","john@gmail.com","john@mail.com"] Đầu ra: ["John","john@gmail.com","john@mail.com"]	❑ Map email -> index ❑ Union các email cùng tài khoản ❑ Nhóm theo phần tử gốc	<pre>class DSU { int[] parent, rank; DSU(int n) { parent = new int[n]; rank = new int[n]; } int find(int x) { return parent[x] == x ? x : find(parent[x]); } void union(int x, int y) { int px = find(x), py = find(y); if (rank[px] < rank[py]) parent[px] = py; else if (rank[px] > rank[py]) parent[py] = px; else { parent[py] = px; rank[px]++; } } public List<List<String>> accountsMerge(List<List<String>> accs) { Map<String, Integer> map = new HashMap<>(); int len = accs.size(); for (List<String> acc : accs) { DSU dsu = new DSU(len); // gộp email trùng nhau sang union, sau đó nhóm theo find(x) return new ArrayList<>(); } } }</pre>	Time: $O(N^2)$ $(N = \text{số tài khoản, Space: } O(N))$ Yikes! DSU gộp email cùng tài khoản để nhóm (a = người tạo Ackermann).
63 Expression Add Operators Chèn toán tử (+, -, *, /) giữa các chữ số để biểu thức bằng target. Đầu vào: num = "123", target = 6 Đầu ra: ["1+2+3", "1*2+3"]	❑ Backtracking ❑ Theo dõi toàn hàng cuối (path) ❑ Xây biểu thức từng bước	<pre>public List<String> addOperators(String num, int target) { List<String> res = new ArrayList<>(); backtrack(num, target, 0, 0, "", res); return res; } private void backtrack(String num, int target, int len, long eval, long last, String path, List<String> res) { if (len == num.length()) { if (eval == target) res.add(path); return; } for (int i = len; i < num.length(); i++) { if (i > len && num.charAt(len) == '0') break; long cur = Long.parseLong(num.substring(len, i+1)); String s = num.substring(len, i+1); if (len == 0) { backtrack(num, target, i+1, cur, cur, res); } else { backtrack(num, target, i+1, eval + cur, cur, path+"+", res); backtrack(num, target, i+1, eval - cur, cur, path+"-", res); backtrack(num, target, i+1, eval * last, last * cur, path+"*", res); backtrack(num, target, i+1, eval / last, last / cur, path+"/", res); } } }</pre>	Time: $O(4^N)$ $(N = \text{số số num, Space: } O(N))$ Yikes! Tạo mỗi vị trí, ta thử rồi đưa 3 toán tử (+, -, /) lần đầu để tránh lỗi.
64 Palindrome Partitioning II Chia chuỗi thành các phần palindrome với số lần cắt tối thiểu. Đầu vào: s = "aab" Đầu ra: 1 ["aa","b"]	❑ Tìm chuỗi palindrome ❑ DP: dp[i] = số lần cắt tối thiểu cho phần từ đầu i	<pre>public int minCut(String s) { int n = s.length(); boolean[][] pal = new boolean[n][n]; for (int len = 1; len <= n; len++) { for (int i = 0; i <= n - len; i++) { if (len == 1) pal[i][i] = true; else if (len == 2) pal[i][i+1] = (s.charAt(i) == s.charAt(i+1)); else pal[i][i+len] = (s.charAt(i) == s.charAt(i+len)) && pal[i+1][i+len-1]; } } int[] dp = new int[n]; for (int i = 0; i < n; i++) { if (pal[0][i]) dp[i] = 0; else { dp[i] = i; for (int j = 0; j < i; j++) { if (pal[j+1][i]) dp[i] = Math.min(dp[i], dp[j]+1); } } } return dp[n-1]; }</pre>	Time: $O(n^2)$ Space: $O(n^2)$ Yikes! Tìm trước bảng palindrome rồi áp dụng DP để tìm số lần cắt nhỏ nhất.
65 Distinct Subsequences Đếm số subsequences khác nhau của s bằng t. Đầu vào: s = "abab", t = "abab" Đầu ra: 3	❑ 2D DP ❑ dp[i][j] = số cách ký tự đầu của s tạo thành ký tự đầu của t ❑ Lây / không lấy ký tự	<pre>public int numDistinct(String s, String t) { int n = s.length(), m = t.length(); int[][] dp = new int[n+1][m+1]; for (int i = 0; i <= n; i++) dp[i][0] = 1; for (int j = 1; j <= m; j++) { for (int i = 1; i <= n; i++) { if (s.charAt(i-1) == t.charAt(j-1)) dp[i][j] = dp[i-1][j] + dp[i-1][j-1]; else dp[i][j] = dp[i-1][j]; } } return dp[n][m]; }</pre>	Time: $O(M * N)$ Space: $O(M * N)$ Yikes! Với mỗi ký tự, ta chọn lấy hoặc không lấy.

Kiên Trì
Bền Bỉ

★ QUICK 2 KNOWLEDGE ★

CẨM NANG TẤT CẢ CÁC PATTERN DSA

Những Điều Bạn Cần Biết Trước Khi Phỏng Vấn

Trang: 13

Ngày: __/__/__



Pattern / Problem	Example	Code (Java)	Time & Space
66 Longest Increasing Subsequence (LIS) Tìm độ dài subsequence tăng dần dài nhất. Đầu vào: [10,9,2,5,3,7,101,18] Đầu ra: 4 (2,3,7,101)	- DP với binary search (n log n) - Duy trì mảng tail - Thay thế / mở rộng	<pre>public int lengthOfLIS(int[] nums) { int n = nums.length; int[] tail = new int[n]; // tail[i] = giá trị // nhỏ nhất int len = 0; for (int x : nums) { int i = 0, j = len; while (i < j) { int m = (i + j) / 2; if (tail[m] < x) i = m + 1; else j = m; } tail[i] = x; if (i == len) len++; } return len; }</pre>	Time: $O(n \log n)$ Space: $O(n)$ Vì sao? Binary search giữ mảng tail nhỏ nhất → LIS tối ưu trong $O(n \log n)$.
67 Longest Common Subsequence (LCS) Tìm độ dài LCS giữa hai chuỗi. Đầu vào: text1 = "abcde", text2 = "ace" Đầu ra: 3 ("ace")	2D DP - Nếu khớp → 1 + đường chéo - Nếu không → max(trên, trái)	<pre>public int longestCommonSubsequence(String a, String b) { int n = a.length(), m = b.length(); int[][] dp = new int[n+1][m+1]; for (int i = 1; i <= n; i++) { for (int j = 1; j <= m; j++) { if (a.charAt(i-1) == b.charAt(j-1)) dp[i][j] = 1 + dp[i-1][j-1]; else dp[i][j] = Math.max(dp[i-1][j], dp[i][j-1]); } } return dp[n][m]; }</pre>	Time: $O(n * m)$ Space: $O(n * m)$ Vì sao? Ta điền bảng kích thước (n+1) x (m+1).
68 0/1 Knapsack Tối đa hóa giá trị với giới hạn trọng lượng. Đầu vào: wt=[1,3,4,5], val=[1,4,5,7], W=7 Đầu ra: 9	2D DP - Lấy hoặc bỏ phần tử - dp[i][w] = giá trị tối nhất dùng i phần tử đầu với trọng lượng w	<pre>public int knapSack(int W, int[] wt, int[] val) { int n = wt.length; int[][] dp = new int[n+1][W+1]; for (int i = 1; i <= n; i++) { for (int w = 0; w <= W; w++) { if (wt[i-1] <= w) dp[i][w] = Math.max(val[i-1] + dp[i-1][w - wt[i-1]], dp[i-1][w]); else dp[i][w] = dp[i-1][w]; } } return dp[n][W]; }</pre>	Time: $O(n * W)$ Space: $O(n * W)$ Vì sao? Với mỗi phần tử và trọng lượng, ta tính giá trị tốt nhất.
69 Coin Change - Min Coins Tìm số lượng xu tối thiểu để đạt amount. Đầu vào: coins=[1,2,5], amount=11 Đầu ra: 3 (5+5+1)	1D DP - dp[i] = số xu tối thiểu cho amount i - Thứ tất cả các đồng xu	<pre>public int coinChange(int[] coins, int amount) { int[] dp = new int[amount+1]; Arrays.fill(dp, amount + 1); dp[0] = 0; for (int i = 1; i <= amount; i++) { for (int c : coins) { if (i >= c) dp[i] = Math.min(dp[i], 1 + dp[i-c]); } } return dp[amount] > amount ? -1 : dp[amount]; }</pre>	Time: $O(\text{amount} * n)$ Space: $O(\text{amount})$ Vì sao? Với mỗi giá trị amount, ta thử mọi đồng xu.
70 Coin Change - Number of Ways Đếm số cách để tạo thành amount. Đầu vào: coins=[1,2,5], amount=5 Đầu ra: 4	1D DP - Duyệt đồng xu trước - dp[i] += dp[i-coin]	<pre>public int change(int amount, int[] coins) { int[] dp = new int[amount+1]; dp[0] = 1; // một cách để tạo 0 for (int c : coins) { for (int i = c; i <= amount; i++) { dp[i] += dp[i - c]; } } return dp[amount]; }</pre>	Time: $O(\text{amount} * n)$ Space: $O(\text{amount})$ Vì sao? Ta đếm tổ hợp, không phải hoán vị.
71 Matrix Chain Multiplication Tìm số phép nhân vô hướng nhỏ nhất để nhân chuỗi ma trận. Đầu vào: p=[40,20,30,10,30] Đầu ra: 26000	- Interval DP - Thứ mọi điểm chia k - dp[i][j] = chi phí nhỏ nhất cho i..j	<pre>public int matrixChainOrder(int[] p) { int n = p.length; int[][] dp = new int[n][n]; for (int len = 2; len < n; len++) { for (int i = 1; i < n - len + 1; i++) { int j = i + len - 1; dp[i][j] = Integer.MAX_VALUE; for (int k = i; k < j; k++) { int cost = dp[i][k] + dp[k+1][j] + p[i-1]*p[k]*p[j]; dp[i][j] = Math.min(dp[i][j], cost); } } } return dp[1][n-1]; }</pre>	Time: $O(n^3)$ Space: $O(n^2)$ Vì sao? Ta thử mọi điểm chia k cho mỗi khoảng (i, j).
72 Burst Balloons Số coin tối đa khi nổ bóng bay tối ưu. Đầu vào: [3,1,5,8] Đầu ra: 167	- Interval DP - Coi bóng cuối nổ trong (i, j) - Thêm vào giá trị 1 ở hai đầu	<pre>public int maxCoins(int[] nums) { int n = nums.length; int[] a = new int[n+2]; a[0] = a[n+1] = 1; for (int i = 1; i <= n; i++) a[i] = nums[i-1]; int[][] dp = new int[n+2][n+2]; for (int len = 1; len <= n; len++) { for (int i = 1; i <= n - len + 1; i++) { int j = i + len - 1; for (int k = i; k < j; k++) { dp[i][j] = Math.max(dp[i][j], dp[i][k-1] + a[i-1]*a[k]*a[j+1] + dp[k+1][j]); } } } return dp[1][n]; }</pre>	Time: $O(n^3)$ Space: $O(n^2)$ Vì sao? Ta coi bóng k là bóng cuối cùng nổ trong (i, j) rồi giải bằng đệ quy.



Pattern / Problem	Example	Code (Java)	Time & Space
73 Maximum Sum Contiguous Subarray (Kadane's Algorithm) Tìm tổng lớn nhất của subarray liên tiếp. Đầu vào: [-2,1,-3,4,-1,2,1,-5,4] Đầu ra: 6 ([4,-1,2,1])	☒ Duy trì tổng hiện tại ☒ Cập nhật tổng lớn nhất ☒ Reset tổng hiện tại khi < 0	<pre>public int maxSubArray(int[] nums) { int max = nums[0], cur = 0; for (int n : nums) { cur += n; max = Math.max(max, cur); if (cur < 0) cur = 0; } return max; }</pre>	Time: O(n) Space: O(1) Vì sao? Ta giữ tổng chạy và reset khi nó âm.
74 Best Time to Buy and Sell Stock Tìm lợi nhuận tối đa khi thực hiện một giao dịch. Đầu vào: [7,1,5,3,6,4] Đầu ra: 5 (Mua=1, Bán=6)	☒ Theo dõi giá nhỏ nhất tính đến hiện tại ☒ Tính lợi nhuận lớn nhất mỗi ngày	<pre>public int maxProfit(int[] prices) { int min = prices[0], profit = 0; for (int p : prices) { min = Math.min(min, p); profit = Math.max(profit, p - min); } return profit; }</pre>	Time: O(n) Space: O(1) Vì sao? Ta luôn giữ giá mua thấp nhất và lợi nhuận tốt nhất tính đến hiện tại.
75 Contains Duplicate Kiểm tra mảng có chứa phần tử trùng không. Đầu vào: [1,2,3,1] Đầu ra: true	☒ Dùng HashSet ☒ Nếu đã tồn tại, trả về true	<pre>public boolean containsDuplicate(int[] nums) { Set<Integer> set = new HashSet<>(); for (int n : nums) { if (set.contains(n)) return true; set.add(n); } return false; }</pre>	Time: O(n) Space: O(n) Vì sao? HashSet giúp tra cứu O(1) để phát hiện trùng lặp.
76 Two Sum Tìm hai số có tổng bằng target. Đầu vào: nums = [2,7,11,15], target = 9 Đầu ra: [0, 1]	☒ Dùng HashMap (value - chỉ số) ☒ Kiểm tra phần bù có tồn tại không	<pre>public int[] twoSum(int[] nums, int target) { Map<Integer, Integer> map = new HashMap<>(); for (int i = 0; i < nums.length; i++) { int complement = target - nums[i]; if (map.containsKey(complement)) return new int[] {map.get(complement), i}; map.put(nums[i], i); } return new int[]{}; }</pre>	Time: O(n) Space: O(n) Vì sao? Ta lưu số đã thấy và kiểm tra phần bù trong O(1).
77 Valid Anagram Kiểm tra hai chuỗi có phải anagram không. Đầu vào: s = "anagram", t = "nagaram" Đầu ra: true	☒ Đếm tần suất ký tự ☒ So sánh hai mảng tần suất	<pre>public boolean isAnagram(String s, String t) { if (s.length() != t.length()) return false; int[] cnt = new int[26]; for (char c : s.toCharArray()) cnt[c - 'a']++; for (char c : t.toCharArray()) cnt[c - 'a']--; for (int x : cnt) if (x != 0) return false; return true; }</pre>	Time: O(n) Space: O(1) Vì sao? Ta so sánh tần suất ký tự của cả hai chuỗi.
78 Product of Array Except Self Trả về mảng mà mỗi phần tử là tích của tất cả phần tử khác. Đầu vào: [1,2,3,4] Đầu ra: [24,12,8,6]	☒ Tích Prefix & Suffix ☒ Không dùng phép chia	<pre>public int[] productExceptSelf(int[] nums) { int n = nums.length; int[] res = new int[n]; res[0] = 1; for (int i = 1; i < n; i++) res[i] = res[i-1] * nums[i-1]; int suf = 1; for (int i = n-1; i >= 0; i--) { res[i] *= suf; suf *= nums[i]; } return res; }</pre>	Time: O(n) Space: O(1) extra Vì sao? Ta dùng tích prefix và suffix để tránh phép chia.
79 Next Permutation Tìm hoán vị kế tiếp theo thứ tự từ điển. Đầu vào: [1,2,3] Đầu ra: [1,3,2]	1. Bước 1: Tìm điểm gãy 2. Bước 2: Tìm số lớn hơn kế tiếp 3. Bước 3: Đảo ngược hậu tố	<pre>public void nextPermutation(int[] nums) { int i = nums.length - 2; while (i >= 0 && nums[i] >= nums[i+1]) i--; if (i >= 0) { int j = nums.length - 1; while (nums[j] <= nums[i]) j--; swap(nums, i, j); } reverse(nums, i+1, nums.length-1); } private void swap(int[] a, int i, int j) { int t = a[i]; a[i] = a[j]; a[j] = t; } private void reverse(int[] a, int l, int r) { while (l < r) swap(a, l++, r--); }</pre>	Time: O(n) Space: O(1) Vì sao? Ta tìm số lớn hơn kế tiếp theo thứ tự từ điển.
80 Longest Consecutive Sequence Tìm độ dài dãy số liên tiếp dài nhất. Đầu vào: [100,4,200,1,3,2]	☒ Dùng HashSet ☒ Chỉ bắt đầu dãy từ số nhỏ nhất (num-1 không có trong set)	<pre>public int longestConsecutive(int[] nums) { Set<Integer> set = new HashSet<>(); for (int n : nums) set.add(n); int best = 0; for (int n : set) { if (!set.contains(n-1)) { int cur = n, len = 1; while (set.contains(cur+1)) { cur++; len++; } best = Math.max(best, len); } } return best; }</pre>	Time: O(n) Space: O(n) Vì sao? Ta chỉ tạo dãy từ điểm bắt đầu để tránh xử lý trùng.



★ QUICK 2 KNOWLEDGE ★

CẨM NANG TẤT CẢ CÁC PATTERN DSA

Những Điều Bạn Cần Biết Trước Khi Phỏng Vấn

Trang: 15

Ngày: _/_/_



Pattern / Problem	Example	Code (Java)	Time & Space
81 Merge Overlapping Intervals Gộp tất cả các interval chồng lấn trong danh sách. Đầu vào: [[1,3],[2,6],[8,10],[15,18],[17,20]] Đầu ra: [[1,6],[8,10],[15,20]]	☒ Sắp xếp theo thời điểm bắt đầu ☒ Gộp khi chồng lấn ☒ Thêm interval không chồng lấn	<pre>public int[][] merge(int[][] intervals) { if (intervals.length == 0) return new int[0][0]; Arrays.sort(intervals, (a,b) -> a[0] - b[0]); List<int[]> res = new ArrayList<>(); int[] cur = intervals[0]; for (int i = 1; i < intervals.length; i++) { if (intervals[i][0] <= cur[1]) { cur[1] = Math.max(cur[1], intervals[i][1]); } else { res.add(cur); cur = intervals[i]; } } res.add(cur); return res.toArray(new int[res.size()][2]); }</pre>	Time: $O(n \log n)$ Space: $O(n)$ Vì sao? Sắp xếp tốn $O(n \log n)$ và gộp tốn $O(n)$.
82 Insert Interval Chèn interval mới vào danh sách interval không chồng lấn đã sắp xếp. Đầu vào: intervals = [[1,3],[6,9]], newInterval = [2,5] Đầu ra: [[1,5],[6,9]]	☒ Thêm tất cả interval trước newInterval ☒ Gộp những cái chồng lấn ☒ Thêm phần còn lại	<pre>public int[][] insert(int[][] intervals, int[] newInt) { List<int[]> res = new ArrayList<>(); int i = 0, n = intervals.length; while (i < n && intervals[i][1] < newInt[0]) res.add(intervals[i++]); while (i < n && intervals[i][0] <= newInt[1]) { newInt[0] = Math.min(newInt[0], intervals[i][0]); newInt[1] = Math.max(newInt[1], intervals[i][1]); i++; } res.add(newInt); while (i < n) res.add(intervals[i++]); return res.toArray(new int[res.size()][2]); }</pre>	Time: $O(n)$ Space: $O(n)$ Vì sao? Ta duyệt một lần và gộp ngay khi gặp.
83 Meeting Rooms II (Minimum Meeting Rooms) Tìm số phòng họp tối thiểu cần thiết. Đầu vào: [[0,30],[5,10],[15,20]] Đầu ra: 2	☒ Sắp xếp theo thời điểm bắt đầu ☒ Dùng min-heap các thời điểm kết thúc ☒ Tái sử dụng phòng khi có thể	<pre>public int minMeetingRooms(int[][] intervals) { Arrays.sort(intervals, (a,b) -> a[0] - b[0]); PriorityQueue<Integer> pq = new PriorityQueue<>(); for (int[] in : intervals) { if (!pq.isEmpty() && pq.peek() <= in[0]) pq.poll(); pq.offer(in[1]); } return pq.size(); }</pre>	Time: $O(n \log n)$ Space: $O(n)$ Vì sao? Sắp xếp là $O(n \log n)$ và thao tác heap cho mỗi cuộc họp.
84 K Closest Points to Origin Tìm k điểm gần gốc tọa độ (0,0) nhất. Đầu vào: points = [[1,3],[-2,2]], k = 1 Đầu ra: [[-2,2]]	☒ Dùng max-heap kích thước k ☒ Lưu (khoảng cách, điểm) ☒ Giữ lại k điểm gần nhất	<pre>public int[][] kClosest(int[][] points, int k) { PriorityQueue<int[]> pq = new PriorityQueue<>((a,b) -> Integer.compare(b[0], a[0])); // max-heap for (int[] p : points) { int dist = p[0]*p[0] + p[1]*p[1]; pq.offer(new int[]{dist, p[0], p[1]}); if (pq.size() > k) pq.poll(); } int[][] res = new int[k][2]; for (int i = k-1; i >= 0; i--) { int[] top = pq.poll(); res[i][0] = top[1]; res[i][1] = top[2]; } return res; }</pre>	Time: $O(n \log k)$ Space: $O(k)$ Vì sao? Ta duy trì heap kích thước k, mỗi thao tác tốn $O(\log k)$.
85 Smallest Range Covering Elements from K Lists Tìm khoảng nhỏ nhất chứa ít nhất một phần tử từ mỗi list. Đầu vào: [[4,10,15,24,26],[0,9,12,20],[5,18,22,30]] Đầu ra: [20, 24]	☒ Dùng min-heap các phần tử đầu ☒ Mở rộng khoảng bằng cách di chuyển con trỏ min ☒ Theo dõi khoảng tốt nhất hiện tại	<pre>public int[] smallestRange(List<List<Integer>> nums) { PriorityQueue<int[]> pq = new PriorityQueue<>((a,b) -> a[0] - b[0]); int max = Integer.MIN_VALUE; int k = nums.size(); for (int i = 0; i < k; i++) { pq.offer(new int[]{nums.get(i).get(0), i, max = Math.max(max, nums.get(i).get(0))}); } int start = 0, end = Integer.MAX_VALUE; while (true) { int[] cur = pq.poll(); // {value, listIndex, idx} int val = cur[0], li = cur[1], idx = cur[2]; if (max - val < end - start) { start = val; end = max; } if (idx + 1 == nums.get(li).size()) break; int next = nums.get(li).get(idx + 1); pq.offer(new int[]{next, li, idx + 1}); max = Math.max(max, next); } return new int[]{start, end}; }</pre>	Time: $O(N \log k)$ (N = tổng số phần tử) Space: $O(k)$ Vì sao? Ta push/pop khỏi heap N lần, kích thước heap là k.



★ QUICK 2 KNOWLEDGE ★

CẨM NANG TẤT CẢ CÁC PATTERN DSA

Những Điều Bạn Cần Biết Trước Khi Phòng Vấn

Trang: 16

Ngày: __/__/__



Pattern / Problem	Example	Code (Java)	Time & Space
86 Rotate Array Xoay mảng sang phải k bước. Đầu vào: nums = [1,2,3,4,5,6,7], k = 3 Đầu ra: [5,6,7,1,2,3,4]	☒ Đảo ngược toàn bộ mảng ☒ Đảo ngược k phần tử đầu ☒ Đảo ngược phần còn lại	<pre>public void rotate(int[] nums, int k) { int n = nums.length; k = k % n; reverse(nums, 0, n - 1); reverse(nums, 0, k - 1); reverse(nums, k, n - 1); } private void reverse(int[] a, int l, int r) { while (l < r) { int temp = a[l]; a[l++] = a[r]; a[r--] = temp; } }</pre>	Time: O(n) Space: O(1) → In-place Vì sao? Ba lần đảo ngược mất thời gian tuyến tính, thực hiện tại chỗ với O(1) bộ nhớ phụ.
87 Product Except Self Trả về mảng mà mỗi phần tử là tích của tất cả phần tử khác. Đầu vào: [1,2,3,4] Đầu ra: [24,12,8,6]	☒ Duyệt tích prefix ☒ Duyệt tích suffix ☒ Không dùng phép chia	<pre>public int[] productExceptSelf(int[] nums) { int n = nums.length; int[] res = new int[n]; res[0] = 1; for (int i = 1; i < n; i++) res[i] = res[i-1] * nums[i-1]; int suffix = 1; for (int i = n-1; i >= 0; i--) { res[i] *= suffix; suffix *= nums[i]; } return res; }</pre>	Time: O(n) Space: O(1) extra (ngoại trừ mảng output) Vì sao? Ta xây tích prefix và suffix trong hai lượt duyệt mà không dùng phép chia.
88 Sliding Window Maximum Tìm giá trị lớn nhất trong mỗi window kích thước k. Đầu vào: nums = [1,3,-1,-3,5,3,6,7], k = 3 Đầu ra: [3,3,5,5,6,7]	☒ Dùng Deque (theo chỉ số) ☒ Loại chỉ số ngoài window ☒ Duy trì thứ tự giảm dần	<pre>public int[] maxSlidingWindow(int[] nums, int k) { if (nums.length == 0) return new int[0]; Deque<Integer> dq = new ArrayDeque<>(); int[] res = new int[nums.length - k + 1]; int idx = 0; for (int i = 0; i < nums.length; i++) { while (!dq.isEmpty() && dq.peekFirst() <= i - k) dq.pollFirst(); while (!dq.isEmpty() && nums[dq.peekLast()] <= nums[i]) dq.pollLast(); dq.offerLast(i); if (i >= k - 1) res[idx++] = nums[dq.peekFirst()]; } return res; }</pre>	Time: O(n) Space: O(k) Vì sao? Mỗi chỉ số được thêm và loại tối đa một lần khỏi deque.
89 Trapping Rain Water Tính lượng nước mưa có thể chứa được. Đầu vào: [0,1,0,2,1,0,1,3,2,1,2,1] Đầu ra: 6	☒ Two pointer approach ☒ Theo dõi leftMax & rightMax ☒ Di chuyển con trỏ ở phía thấp hơn	<pre>public int trap(int[] height) { int l = 0, r = height.length - 1; int leftMax = 0, rightMax = 0, water = 0; while (l < r) { if (height[l] < height[r]) { leftMax = Math.max(leftMax, height[l]); water += leftMax - height[l]; l++; } else { rightMax = Math.max(rightMax, height[r]); water += rightMax - height[r]; r--; } } return water; }</pre>	Time: O(n) Space: O(1) Vì sao? Ta di chuyển con trỏ ở phía thấp hơn và tính nước dựa trên biến tốt nhất.
90 Evaluate Reverse Polish Notation (RPN) Tính giá trị của biểu thức số học ở dạng RPN. Đầu vào: ["2","1","+","3","*"] Đầu ra: 9	☒ Dùng Stack ☒ Nếu là toán tử → pop hai giá trị ☒ Push kết quả trở lại	<pre>public int evalRPN(String[] tokens) { Stack<Integer> st = new Stack<>(); for (String t : tokens) { if (isOp(t)) { int b = st.pop(); int a = st.pop(); st.push(apply(a, b, t)); } else { st.push(Integer.parseInt(t)); } } return st.pop(); } private boolean isOp(String s) { return s.equals("+") s.equals("-") s.equals("*") s.equals("/"); } private int apply(int a, int b, String op) { if (op.equals("+")) return a + b; if (op.equals("-")) return a - b; // ... return 0; }</pre>	Time: O(n) Space: O(n) Vì sao? Ta xử lý mỗi token một lần và stack chứa toán hạng trong trường hợp xấu nhất.



Pattern / Problem	Example	Code (Java)	Time & Space
91 Find Missing Number Cho n số từ 0 đến n, tìm số bị thiếu. Đầu vào: [3,0,1], n = 3 Đầu ra: 2	☑ Dùng công thức Gauss: $sum = n * (n+1) / 2$ ☑ Lấy tổng trừ đi tổng thực tế ☑ XOR approach cũng dùng được	<pre>public int missingNumber(int[] nums, int n) { int total = n * (n + 1) / 2; int sum = 0; for (int num : nums) sum += num; return total - sum; } // XOR Approach public int missingNumberXOR(int[] nums, int n) { int xor = n; for (int i = 0; i < nums.length; i++) { xor ^= i; xor ^= nums[i]; } return xor; }</pre>	Time: O(n) Space: O(1) Vì sao? Tổng (0..n) trừ đi tổng thực tế cho ra số bị thiếu trong O(n).
92 Find Duplicate Number Tìm số bị trùng trong mảng chứa n+1 số nguyên. Các số nằm từ 1 đến n. Đầu vào: [1,3,4,2,2] Đầu ra: 2	☑ Floyd's Tortoise and Hare (Cycle Detection) ☑ Xem mảng như linked list	<pre>public int findDuplicate(int[] nums) { int slow = nums[0], fast = nums[0]; do { slow = nums[slow]; fast = nums[nums[fast]]; } while (slow != fast); fast = nums[0]; while (slow != fast) { slow = nums[slow]; fast = nums[fast]; } return slow; }</pre>	Time: O(n) Space: O(1) Vì sao? Chu trình tồn tại vì có số trùng. Điểm bắt đầu chu trình chính là số trùng.
93 Count Inversions (Merge Sort) Đếm cặp (i, j) sao cho i < j và arr[i] > arr[j]. Đầu vào: [2,4,1,3,5] Đầu ra: 3 (Cặp: (2,1), (4,1), (4,3))	☑ Sửa Merge Sort ☑ Khi arr[i] > arr[j], mọi phần tử còn lại bên trái đều là inversion	<pre>public long countInversions(int[] arr) { int[] temp = new int[arr.length]; return mergeSort(arr, 0, arr.length-1, temp); } private long mergeSort(int[] arr, int l, int r, int[] temp) { if (l >= r) return 0; int m = l + (r - l) / 2; long inv = mergeSort(arr, l, m, temp) + mergeSort(arr, m+1, r, temp); inv += merge(arr, l, m, r, temp); return inv; } // merge() đếm inversion trong khi gộp</pre>	Time: O(n log n) Space: O(n) Vì sao? Bước merge giúp đếm inversion hiệu quả.
94 Subarray with Given Sum Tìm số lượng subarray có tổng bằng k. Đầu vào: [1,1,1], k = 2 Đầu ra: 2 ([1,1], [1,1])	☑ Dùng Prefix Sum + HashMap ☑ Lưu tần suất prefixSum	<pre>public int subarraySum(int[] nums, int k) { Map<Integer, Integer> map = new HashMap<>(); map.put(0, 1); int sum = 0, count = 0; for (int num : nums) { sum += num; if (map.containsKey(sum - k)) count += map.get(sum - k); map.put(sum, map.getOrDefault(sum, 0) + 1); } return count; }</pre>	Time: O(n) Space: O(n) Vì sao? Prefix sum giúp ta biết (sum - k) đã xuất hiện bao nhiêu lần trước đó.
95 Implement LRU Cache Thiết kế LRU Cache với get(key) và put(key, value) trong O(1). Capacity = 2 Operations: put(1,1), put(2,2), get(1), put(3,3), get(2) Đầu ra: [1, -1]	1. Dùng HashMap + Doubly Linked List 2. Head = phần tử vừa dùng gần đây nhất (MRU) 3. Tail = phần tử ít dùng nhất (LRU) 4. Khi get/put: di chuyển node lên đầu 5. Khi đầy: xóa tail	Idea: - HashMap: key → node (truy cập O(1)) - Doubly Linked List: duy trì thứ tự sử dụng - Khi get/put: di chuyển node lên đầu (most recent) - Khi đầy: xóa node ở tail (least recent)	Time: O(1) Space: O(Capacity) Vì sao? HashMap giúp truy cập trực tiếp và DLL duy trì thứ tự hoàn hảo.

🎯 TỔNG KẾT - ĐIỀU CỐT LỖI

- ☑ Nắm vững các pattern này. Chúng bao phủ hơn 80% câu hỏi DSA.
- ☑ Tập trung vào hiểu bản chất, không chỉ học thuộc code.
- ☑ Luyện tập hàng ngày - sự đều đặn quan trọng hơn động lực nhất thời.

⌚ TIME COMPLEXITY CHEAT SHEET

- O(1) - Hằng số
- O(log n) - Logarit
- O(n) - Tuyến tính

🧠 SPACE COMPLEXITY CHEAT SHEET

- O(1) - Hằng số (không tốn thêm bộ nhớ)
- O(log n) - Đệ quy stack / log
- O(n) - Tuyến tính, thêm bộ nhớ

Kiên Trì
Bền Bì



★ QUICK 2 KNOWLEDGE ★

CẨM NANG TẤT CẢ CÁC PATTERN DSA

Những Điều Bạn Cần Biết Trước Khi Phỏng Vấn

≈ TỔNG KẾT CUỐI CÙNG - NHỮNG GÌ CHÚNG TA ĐÃ HỌC ≈

Trang: 18 (Tổng kết)

Ngày: __/__/__



Chúng ta đã đi qua **95 pattern DSA quan trọng nhất** cùng với bài toán, ví dụ, cách tiếp cận, code Java, và độ phức tạp về thời gian/bộ nhớ. Những pattern này xuất hiện trong hầu hết mọi buổi phỏng vấn coding. Nắm vững chúng và bạn đã sẵn sàng **80%! 100**

Ghi nhớ

- ☒ Pattern > Học thuộc lòng
- ☒ Hiểu > Chỉ viết code
- ☒ Luyện tập > Hoàn hảo
- ☒ Đều đặn > Tài năng ♥

🔑 ĐIỀU CỐT LÕI CẦN NHỚ

- ☒ Pattern giúp bạn tư duy đúng hướng.
- ☒ Nhận diện pattern từ đề bài.
- ☒ Chọn cách tiếp cận tốt nhất.
- ☒ Giải quyết từng bước với code sạch.
- ☒ Phân tích Time & Space Complexity.
- ☒ Dry run với ví dụ cụ thể.
- ☒ Viết code modular, dễ đọc.
- ☒ Test với edge case.
- ☒ Ôn tập thường xuyên.
- ☒ Áp dụng vào contest & bài toán thực tế.

🔍 HƯỚNG DẪN NHẬN DIỆN PATTERN

Arrays / Strings → Sliding Window, Two Pointers, Prefix Sum, Subarray, v.v.

Linked List → Fast & Slow, Reverse, Merge, Cycle Detection, v.v.

Stack / Queue → Valid Parentheses, RPN, Next Greater, Monotonic Stack, v.v.

Trees → DFS, BFS, LCA, Serialize, Diameter, Views, v.v.

Graphs → BFS, DFS, Dijkstra, Topo Sort, Union Find, v.v.

DP → 1D DP, 2D DP, Knapsack, LIS, LCS, v.v.

Heaps → Kth Element, Merge K lists, Top K frequent, v.v.

Backtracking → Subsets, Permutations, N-Queens, Sudoku, v.v.

📄 COMPLEXITY CHEAT SHEET

TIME COMPLEXITY (Phổ biến)

$O(1)$ - Hằng số
 $O(\log n)$ - Logarit
 $O(n)$ - Tuyến tính
 $O(n \log n)$ - Linearithmic
 $O(n^2)$ - Bậc hai
 $O(\log n)^2$ - Log bình phương
 $O(2^n)$ - Mũ } **Nên tránh**
 $O(n!)$ - Giai thừa

SPACE COMPLEXITY (Phổ biến)

$O(1)$ - Hằng số (không tốn thêm bộ nhớ)
 $O(n)$ - Tuyến tính
 $O(\log n)$ - Độ quy stack / log
 $O(n \log n)$ - Merge sort, v.v.
 $O(n^2)$ - 2D DP / Matrix

🎯 LUYỆN TẬP HIỆU QUẢ

- Chọn một pattern.
- Hiểu rõ trực giác đằng sau nó.
- Giải từ dễ → trung bình → khó.
- Dry run nhiều ví dụ.
- Viết code tối ưu & sạch.
- Phân tích độ phức tạp.
- Ôn lại & giải lại sau vài ngày.
- Áp dụng vào contest & dự án thực tế.

Nguyên Tắc Vàng ★

Đừng chỉ viết code.

Think → Code → Optimize → Review

📋 CHECKLIST TRƯỚC PHỎNG VẤN

- ☒ Ôn lại tất cả pattern và cách tiếp cận chính.
- ☒ Giải các bài được gắn tag theo từng công ty.
- ☒ Thành thạo phân tích time & space.
- ☒ Luyện giải thích rõ ràng tư duy của bạn.
- ☒ Giải bài mà không nhìn lời giải.
- ☒ Xem lại edge case & test case.
- ☒ Giữ bình tĩnh, suy nghĩ từng bước.
- ☒ Giao tiếp rõ ràng trong khi code.

🎯 TƯ DUY ĐỂ THÀNH CÔNG

- ☒ Sai lầm là một phần của việc học.
- ☒ Mỗi bài toán giúp bạn giỏi hơn.
- ☒ Đều đặn thắng cường độ.
- ☒ Tập trung vào hiểu bản chất, không học vẹt.
- ☒ Tin vào chính mình. Bạn làm được!

Cứ tiếp tục. Cứ phát triển. Đừng bao giờ bỏ cuộc!

📅 KẾ HOẠCH HÀNG NGÀY

- ☒ Giải 2-3 bài mỗi ngày
- ☒ Ôn lại một pattern cũ
- ☒ Ghi chú lại lỗi sai thường gặp

📚 TÀI NGUYÊN NÊN DÙNG

- LeetCode
- GeeksforGeeks
- NeetCode

🌟 CHÚC BẠN MAY MẮN! 🌟

Bạn đã chuẩn bị kỹ càng.
Giờ là lúc tỏa sáng! 🌟

Kiên Trì
Bền Bỉ



★ QUICK 2 KNOWLEDGE ★

CẨM NANG TẤT CẢ CÁC PATTERN DSA

Những Điều Bạn Cần Biết Trước Khi Phỏng Vấn

Trang: 19 (Bonus)

Ngày: __/__/__



≈ TRANG BONUS - PATTERN & MẸO BỔ SUNG ≈

Các pattern bonus này cũng xuất hiện trong phỏng vấn.
Biết chúng = Lợi thế thêm! ★

1 Bit Manipulation

Dùng toán tử bitwise (&, |, ^, ~, <<, >>)

Check, set, clear bit

Đếm set bits, kiểm tra lũy thừa của 2

Ví dụ:

Kiểm tra n có phải lũy thừa của 2 không $\rightarrow n \& (n-1) == 0$

2 Math & Geometry

GCD / LCM

Kiểm tra số nguyên tố (Sieve)

Công thức hình học

Ví dụ:

GCD của hai số dùng thuật toán Euclid

3 String Pattern Matching

KMP Algorithm

Rabin-Karp Algorithm

Z-Algorithm

Ví dụ:

Tìm pattern trong văn bản dài một cách hiệu quả

4 Game Theory (Cơ bản)

Trạng thái Thắng / Thua

Nước đi tối ưu

Minimax (mức nhập môn)

Ví dụ:

Nim Game, Stone Game (các biến thể nhỏ)

5 Randomized Algorithms

Xáo trộn ngẫu nhiên (Random shuffle)

Reservoir Sampling

Randomized QuickSort

Ví dụ:

Chọn k phần tử ngẫu nhiên từ một stream

6 Design / System Thinking

Design patterns

Rate limiting, Caching

Load balancing (Cơ bản)

Ví dụ:

Thiết kế Twitter, LRU Cache, Parking Lot

★ MẸO PHỎNG VẤN

- ✓ Làm rõ đề bài trước khi giải.
- ✓ Nói to suy nghĩ & giải thích cách tiếp cận.
- ✓ Bắt đầu bằng brute force, rồi tối ưu.
- ✓ Trao đổi về time & space complexity.
- ✓ Xử lý các edge case.
- ✓ Test code với ví dụ mẫu.
- ✓ Giữ code sạch, dễ hiểu.
- ✓ Giữ bình tĩnh & tự tin!

🕒 TÓM TẮT NHANH ĐỘ PHỨC TẠP

TỐT NHẤT

$O(1)$

Hằng số 😊

TẠM ỔN

$O(n \log n)$

Linearithmic

$O(n^2)$

Bậc hai

TỐT

$O(\log n)$

Logarit 😊

$O(n)$

Tuyến tính 😊

KHÔNG TỐT

$O(n^2)$ trở lên

Bậc ba+ ☹️

🧠 TÓM TẮT SPACE COMPLEXITY

$O(1)$ - Hằng số (không thêm bộ nhớ)
 $O(n)$ - Tuyến tính (Array, HashMap, v.v.)
 $O(\log n)$ - Đệ quy stack (Binary Search)
 $O(n \log n)$ - MergeSort, QuickSort (trung bình)
 $O(n^2)$ - 2D DP / Matrix

Bộ nhớ rất quan trọng với input lớn!
Luôn tối ưu. ★

📖 CÁCH HỌC CÁC PATTERN NÀY

① Học Pattern

Hiểu khái niệm, trực giác và use case.

② Giải EASY

Giải bài đơn giản để xây nền tảng.

③ Luyện MEDIUM

Cải thiện logic & xử lý biến thể.

④ Thử HARD

Thách thức bản thân, tư duy sáng tạo.

⑤ Ôn & Xem lại

Ôn tập đều đặn, đừng quên pattern.

✗ LỖI THƯỜNG GẶP

- ✗ Nhảy vào giải ngay mà không suy nghĩ.
- ✗ Không phân tích độ phức tạp.
- ✗ Bỏ qua edge case.
- ✗ Làm phức tạp hóa bài toán đơn giản.
- ✗ Không ôn tập sau khi học.

🏆 ĐỘNG LỰC CUỐI CÙNG

- ★ DSA không phải chạy nhanh, mà là chạy bền.
- ★ Tiến bộ nhỏ mỗi ngày = kết quả lớn lâu dài.
- ★ Đầu đặn quan trọng hơn cường độ.
- ★ Tin vào quá trình. Thành công là chắc chắn.

BẠN LÀM ĐƯỢC! 🍀

Kiên Trì
Bền Bì



★ QUICK 2 KNOWLEDGE ★

CẨM NANG TẮT CẢ CÁC PATTERN DSA

Những Điều Bạn Cần Biết Trước Khi Phỏng Vấn

Trang: 20 (Tổng kết cuối)

Ngày: __/__/__

≈ TRANG CUỐI - HÀNH TRÌNH CỦA BẠN BẮT ĐẦU TỪ ĐÂY! ≈

Luôn Ghi Nhớ

- ☑ Điều đặn > Động lực
- ☑ Luyện tập > Hoàn hảo
- ☑ Kiên nhẫn > Tốc độ
- ☑ Tập trung > Xao nhãng
- ☑ Việc học không bao giờ dừng



Cẩm nang này đã bao phủ **95 pattern DSA mạnh mẽ** cùng với bài toán, ví dụ, cách tiếp cận, code Java và độ phức tạp về thời gian/bộ nhớ. Nắm vững những pattern này, luyện tập đều đặn và bạn sẽ sẵn sàng cho phỏng vấn và sẵn sàng đi xin việc! **100** ✨

🏆 NHỮNG GÌ BẠN ĐÃ ĐẠT ĐƯỢC

- ☑ 95 Pattern DSA quan trọng
 - ☑ Để bài rõ ràng
 - ☑ Nhiều cách tiếp cận
 - ☑ Code Java (dễ hiểu)
 - ☑ Time & Space Complexity
 - ☑ Mẹo phỏng vấn & edge case
 - ☑ Chiến lược & tư duy luyện tập
 - ☑ Tự tin để chinh phục phỏng vấn!
- Bạn đã sẵn sàng! 🍀

17 CÔNG THỨC THÀNH CÔNG HÀNG NGÀY

1. Chọn 2-3 bài mỗi ngày
2. Ôn lại một pattern cũ
3. Theo dõi tiến độ
4. Phân tích Time & Space
5. Học từ lỗi sai
6. Giữ đều đặn
7. Tin vào bản thân

Bước nhỏ mỗi ngày
Thành công lớn về sau! 🌱

📊 THEO DÕI TIẾN ĐỘ

Tuần	Trọng tâm	Số bài	Đã ôn
Tuần 1			
Tuần 2			
Tuần 3			
Tuần 4			

Tự chấm điểm:



(1 = Cần cố gắng, 5 = Xuất sắc)

★ TỔNG HỢP THEO NHÓM PATTERN (Danh mục chính)

Arrays / Strings

- Sliding Window
- Two Pointers
- Prefix Sum
- Subarray / Substring
- In-place Operations

Linked List

- Fast & Slow
- Reverse
- Merge Lists
- Cycle Detection
- LRU / Design

Stack / Queue

- Valid Parentheses
- Monotonic Stack
- Next Greater
- Queue / Deque
- RPN / Min Stack

Trees / Graphs

- DFS / BFS
- Diameter
- Topo Sort
- Union Find
- Tree Traversal

DP / Recursion

- 1D / 2D DP
- Memoization
- Knapsack
- LCS / LIS
- Recursion Trees

Heaps / Backtracking

- Heap Operations
- Kth Element
- Subsets / Permutations
- N-Queens
- Sudoku / M Coloring

📋 CHECKLIST TRƯỚC KHI BƯỚC VÀO PHỎNG VẤN

- ☑ Ôn lại tất cả pattern và cách tiếp cận chính.
- ☑ Giải các bài được gắn tag theo công ty.
- ☑ Luyện edge case & test case.
- ☑ Phân tích Time & Space Complexity.
- ☑ Truyền đạt rõ tư duy của bạn.
- ☑ Viết code sạch, modular.
- ☑ Dry run trước khi code.
- ☑ Giữ bình tĩnh, suy nghĩ từng bước.
- ☑ Đừng bỏ cuộc, thử nhiều cách.
- ☑ Tự tin! Bạn đã chuẩn bị kỹ!

🧠 TƯ DUY QUAN TRỌNG

- ★ Sai lầm là một phần của việc học.
- ★ Mỗi lần bị từ chối là một sự chuyển hướng.
- ★ So sánh bản thân hôm nay với hôm qua, không phải người khác.
- ★ Tập trung vào chiều sâu, không chỉ chiều rộng.
- ★ Đừng dừng khi mệt, hãy dừng khi xong.

Kỷ Luật Hôm Nay, Thành Công Ngày Mai! 🍀

❌ LỖI THƯỜNG GẶP

- ❌ Bỏ qua nền tảng (Arrays, Strings, Recursion)
- ❌ Chỉ xem, không tự giải
- ❌ Không ôn lại pattern cũ
- ❌ Bỏ qua Time & Space
- ❌ Viết code lộn xộn, chưa tối ưu

📖 TÀI NGUYÊN LUYỆN TẬP TỐT NHẤT

- LeetCode
- GeeksforGeeks
- NeetCode
- Striver A2Z Sheet
- Company Tagged Lists

🚩 ĐỘNG LỰC CUỐI CÙNG

Bạn có sức mạnh để đạt được bất cứ điều gì bạn quyết tâm.

Cứ Học. Cứ Phát Triển. Cứ Code.